

C++ Standard Library Immediate Issues to be moved in Croydon, Mar. 2026

Doc. no. P4146R0

Date: 2026-03-27

Audience: WG21

Reply to: Jonathan Wakely <lwgchair@gmail.com>

- [Immediate Issues](#)
 - [3662](#). `basic_string::append/assign(NTBS, pos, n)` suboptimal
 - [3777](#). Common cartesian_product_view produces an invalid range if the first range is input and one of the ranges is empty
 - [3797](#). elements_view insufficiently constrained
 - [3891](#). LWG 3870 breaks `std::expected<cv T, E>`
 - [4026](#). Assignment operators of `std::expected` should propagate triviality
 - [4122](#). Ill-formed operator<=> can cause hard error when instantiating `std::inplace_vector`
 - [4133](#). `awaitable-receiver`'s members are potentially throwing
 - [4143](#). `execution::set_value/set_error/set_stopped/start` should always return void
 - [4151](#). Precondition of `inplace_vector::swap`
 - [4223](#). Deduction guides for maps are mishandling tuples and references
 - [4314](#). Missing move in `mdspan` layout mapping::operator()
 - [4325](#). `std::indirect`'s operator== still does not support incomplete types
 - [4339](#). task's coroutine frame may be released late
 - [4347](#). task's stop source is always created
 - [4354](#). Reconsider `weakly_parallel` as the default `forward_progress_guarantee`
 - [4361](#). `awaitable-receiver::set_value` should use *Mandates* instead of constraints
 - [4379](#). `hive::reserve()` needs *Throws*: element adjusted to match block min/max considerations
 - [4476](#). `run_loop` should not have a `set_error` completion
 - [4478](#). `meta::has_identifier` is not specified for annotations
 - [4485](#). Move specification for `task::stop_token_type`
 - [4497](#). `std::nullopt_t` should be comparable
 - [4504](#). Wording problem in `{simple}_counting_scope`
 - [4506](#). `source_location` is explicitly unspecified if is `constexpr` or not
 - [4532](#). Imprecise `std::polymorphic` wording seems to imply slicing
 - [4533](#). `not_fn<f>` is unimplementable
 - [4537](#). Improve `define_static_array`
 - [4540](#). `future-sender` s returned from `spawn_future` do not forward stop requests to spawned work
 - [4544](#). Parallel overload of `ranges::set_difference` should return `in_in_out_result`
 - [4548](#). Parallel `ranges::set_intersection` should not do unnecessary work
 - [4549](#). `vprint_nonunicode_buffered` ignores its `stream` parameter
 - [4550](#). Need new feature test macros for `<stdckdint.h>` and `<stdbit.h>`
 - [4552](#). `compare_exchange_weak` writes a value on spurious failure, not memory contents
 - [4553](#). Wording for FR-025-246 25.7.18.2 Add a `reserve_hint` function to `concat_view`
 - [4554](#). Remove undefined behaviour from `hive` for invalid limits
 - [4555](#). Remove `is_consteval_only`
 - [4556](#). Unclear properties of reflection strings
 - [4557](#). Remove `constexpr` from `owner_less` and `owner_before`

Immediate Issues

[3662](#). `basic_string::append/assign(NTBS, pos, n)` suboptimal

Section: 27.4.3.7.2 [\[string.append\]](#), 27.4.3.7.3 [\[string.assign\]](#) Status: [Immediate](#) Submitter: Jonathan Wakely Opened: 2022-01-21 Last modified: 2026-03-23

Priority: 3

View all other [issues](#) in `[string.append]`.

Discussion:

Addresses [RU-268](#)

This will allocate temporary string, then copy one byte from it:

```
std::string s;
s.append("a very very long string that doesn't fit in SS0 buffer", 13, 1);
```

The problem is that there is no overload accepting an NTBS there, so it converts to a temporary string using:

```
append(const basic_string&, size_t, size_t = npos);
```

C++17 added a new overload for a `string_view`:

```
append(const T&, size_t, size_t = npos);
```

But that overload is constrained to not allow `const char*` arguments, so we still create a temporary string.

If we're going to accept those arguments, we should do so efficiently, without forcing an unnecessary allocation.

We could do better if we split the `append(const T&, ...)` overload into:

```
append(const T&, size_t);
append(const T&, size_t, size_t);
```

and then remove the `!is_convertible_v<const T&, const charT*>` constraint from the second overload (**Option A** in the proposed resolution).

Alternatively, we can leave the `append(const T&, size_t, size_t)` overload alone and just add one taking exactly the arguments used above. That seems simpler to me, and I prefer that (**Option B** in the Proposed Resolution).

The same problem applies to `assign(..., pos, n)`.

[2022-01-30; Reflector poll]

Set priority to 3 after reflector poll.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4901](#).

[Drafting Note: Two mutually exclusive options are prepared, depicted below by **Option A** and **Option B**, respectively.]

Option A:

1. Modify 27.4.3.1 [\[basic.string.general\]](#), class template `basic_string` synopsis, as indicated:

```
[...]
// 27.4.3.7 \[string.modifiers\], modifiers
[...]
```

```
template<class T>
constexpr basic_string& append(const T& t);
template<class T>
constexpr basic\_string& append\(const T& t, size\_type pos\);
template<class T>
constexpr basic_string& append(const T& t, size_type pos, size_type n = npos);
constexpr basic_string& append(const charT* s, size_type n);
[...]
```

```
template<class T>
constexpr basic_string& assign(const T& t);
template<class T>
constexpr basic\_string& assign\(const T& t, size\_type pos\);
template<class T>
constexpr basic_string& assign(const T& t, size_type pos, size_type n = npos);
constexpr basic_string& assign(const charT* s, size_type n);
[...]
```

2. Modify 27.4.3.7.2 [\[string.append\]](#) as indicated:

```
template<class T>
constexpr basic_string& append(const T& t);
```

-3- Constraints:

(3.1) — `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and

(3.2) — `is_convertible_v<const T&, const charT*>` is false.

-4- Effects: Equivalent to:

```
basic_string_view<charT, traits> sv = t;
return append(sv.data(), sv.size());
```

```
template<class T>
constexpr basic\_string& append\(const T& t, size\_type pos\);
```

[-?- Constraints:](#)

[\(?1\) — is convertible v<const T&, basic_string_view<charT, traits>> is true and](#)

[\(?2\) — is convertible v<const T&, const charT*> is false.](#)

[-?- Effects: Equivalent to:](#)

```
basic\_string\_view<charT, traits> sv = t;
return append\(sv.substr\(pos\)\);
```

```
template<class T>
constexpr basic_string& append(const T& t, size_type pos, size_type n = npos);
```

-5- Constraints:

~~(5.1) — is_convertible_v<const T&, basic_string_view<charT, traits>> is true and~~

~~(5.2) — is_convertible_v<const T&, const charT*> is false.~~

-6- Effects: Equivalent to:

```
basic_string_view<charT, traits> sv = t;
return append(sv.substr(pos, n));
```

3. Modify 27.4.3.7.3 [\[string.assign\]](#) as indicated:

```
template<class T>
constexpr basic_string& assign(const T& t);
```

-4- Constraints:

(4.1) — is_convertible_v<const T&, basic_string_view<charT, traits>> is true and

(4.2) — is_convertible_v<const T&, const charT*> is false.

-5- Effects: Equivalent to:

```
basic_string_view<charT, traits> sv = t;
return assign(sv.data(), sv.size());
```

```
template<class T>
constexpr basic_string& assign(const T& t, size_type pos);
```

-?- Constraints:

(?1) — is_convertible_v<const T&, basic_string_view<charT, traits>> is true and

(?2) — is_convertible_v<const T&, const charT*> is false.

-?- Effects: Equivalent to:

```
basic_string_view<charT, traits> sv = t;
return assign(sv.substr(pos));
```

```
template<class T>
constexpr basic_string& assign(const T& t, size_type pos, size_type n = npos);
```

-6- Constraints:

~~(6.1) — is_convertible_v<const T&, basic_string_view<charT, traits>> is true and~~

~~(6.2) — is_convertible_v<const T&, const charT*> is false.~~

-7- Effects: Equivalent to:

```
basic_string_view<charT, traits> sv = t;
return assign(sv.substr(pos, n));
```

Option B:

1. Modify 27.4.3.1 [\[basic.string.general\]](#), class template basic_string synopsis, as indicated:

```
[...]
// 27.4.3.7 \[string.modifiers\], modifiers
[...]  
constexpr basic_string& append(const charT* s, size_type n);  
constexpr basic_string& append(const charT* s);  
constexpr basic_string& append(const charT* s, size_type pos, size_type n);  
constexpr basic_string& append(size_type n, charT c);  
[...]  
constexpr basic_string& assign(const charT* s, size_type n);  
constexpr basic_string& assign(const charT* s);  
constexpr basic_string& assign(const charT* s, size_type pos, size_type n);  
constexpr basic_string& assign(size_type n, charT c);  
[...]
```

2. Modify 27.4.3.7.2 [\[string.append\]](#) as indicated:

```
constexpr basic_string& append(const charT* s, size_type n);
```

-7- Preconditions: [s, s + n) is a valid range.

-8- Effects: Appends a copy of the range [s, s + n) to the string.

-9- Returns: *this.

```
constexpr basic_string& append(const charT* s);
```

-10- Effects: Equivalent to: return append(s, traits::length(s));

```
constexpr basic_string& append(const charT* s, size_type pos, size_type n);
```

-?- Effects: Equivalent to: `return append(basic_string_view<charT, traits>(s).substr(pos, n));`

3. Modify 27.4.3.7.3 [\[string.assign\]](#) as indicated:

```
constexpr basic_string& assign(const charT* s, size_type n);
```

-8- Preconditions: `[s, s + n)` is a valid range.

-9- Effects: Replaces the string controlled by `*this` with a copy of the range `[s, s + n)`.

-10- Returns: `*this`.

```
constexpr basic_string& assign(const charT* s);
```

-11- Effects: Equivalent to: `return assign(s, traits::length(s));`

```
constexpr basic_string& assign(const charT* s, size_type pos, size_type n);
```

-?- Effects: Equivalent to: `return assign(basic_string_view<charT, traits>(s).substr(pos, n));`

[2026-03-23 Croydon; accept Option B and move to Immediate]

We would like to make this even more efficient, by not doing `char_traits::length(s)` for the whole thing. That would mean that `s` does not have to be null-terminated. But would need something in `char_traits` like `strnlen`. That's a pre-existing problem though, Option B improves on the status quo even if it's not perfect.

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 27.4.3.1 [\[basic.string.general\]](#), class template `basic_string` synopsis, as indicated:

```
[...]
// 27.4.3.7 \[string.modifiers\], modifiers
[...]
constexpr basic_string& append(const charT* s, size_type n);
constexpr basic_string& append(const charT* s);
constexpr basic_string& append(const charT* s, size_type pos, size_type n);
constexpr basic_string& append(size_type n, charT c);
[...]
constexpr basic_string& assign(const charT* s, size_type n);
constexpr basic_string& assign(const charT* s);
constexpr basic_string& assign(const charT* s, size_type pos, size_type n);
constexpr basic_string& assign(size_type n, charT c);
[...]
```

2. Modify 27.4.3.7.2 [\[string.append\]](#) as indicated:

```
constexpr basic_string& append(const charT* s, size_type n);
```

-7- Preconditions: `[s, s + n)` is a valid range.

-8- Effects: Appends a copy of the range `[s, s + n)` to the string.

-9- Returns: `*this`.

```
constexpr basic_string& append(const charT* s);
```

-10- Effects: Equivalent to: `return append(s, traits::length(s));`

```
constexpr basic_string& append(const charT* s, size_type pos, size_type n);
```

-?- Effects: Equivalent to: `return append(basic_string_view<charT, traits>(s).substr(pos, n));`

3. Modify 27.4.3.7.3 [\[string.assign\]](#) as indicated:

```
constexpr basic_string& assign(const charT* s, size_type n);
```

-8- Preconditions: `[s, s + n)` is a valid range.

-9- Effects: Replaces the string controlled by `*this` with a copy of the range `[s, s + n)`.

-10- Returns: `*this`.

```
constexpr basic_string& assign(const charT* s);
```

-11- Effects: Equivalent to: `return assign(s, traits::length(s));`

```
constexpr basic_string& assign(const charT* s, size_type pos, size_type n);
```

-?- Effects: Equivalent to: `return assign(basic_string_view<charT, traits>(s).substr(pos, n));`

3777. Common cartesian_product_view produces an invalid range if the first range is input and one of the ranges is empty

Section: 25.7.33.2 [\[range.cartesian.view\]](#) Status: [Immediate](#) Submitter: Tomasz Kamiński Opened: 2022-09-12 Last modified: 2026-03-27

Priority: 2

Discussion:

In case when cartesian_product_view is common and one of the inner ranges is empty, it needs to produce equal iterators from begin/end. We currently create a sequence of begin iterators as both begin and end iterators. This assumes that begin iterator is copyable, which may not be the case with the input range, even in the case if that range is common — in such case, we require that only sentinel is semantically copy-constructible, not begin even if they are the same type.

To illustrate, C++98 input iterators (like directory_iterator) are syntactically copy-constructible, but only default constructed object, that corresponds to sentinels are semantically copyable — the copy produces an equivalent result. As a consequence for directory_iterator d, and empty std::string_view sv, the view::cartesian_product(d, sv) produces an invalid range.

To fix the problem, we need to move the logic of adjusting the first range iterator to return [end, begin, ..., begin] for begin. This is safe, as we require the end to be always semantically copy-constructible. This again can be done only if computing the end can be done in $\mathcal{O}(1)$ i.e. the first range is common.

[2022-09-28; Reflector poll]

Set priority to 2 after reflector poll.

[2022-09-28; LWG telecon]

Discussed issue. Tim suggested to add a new semantic requirement to sentinel_for that when S and I are the same type then $i == i$ is true for any non-singular i of type I.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4910](#).

1. Modify 25.7.33.2 [\[range.cartesian.view\]](#) as indicated:

[Drafting note: We can optimize the comparison with default_sentinel_t to compare only the iterator to the first range if the range is common. This is observable, as we call comparison of user-provided iterators.]

```
constexpr iterator<false> begin()
requires (!simple-view<First> || ... || !simple-view<Vs>);
```

~~-2 Effects: Equivalent to: return iterator<false>(tuple-transform(ranges::begin, bases_));~~

```
constexpr iterator<true> begin() const
requires (range<const First> && ... && range<const Vs>);
```

~~-3 Effects: Equivalent to: return iterator<true>(tuple-transform(ranges::begin, bases_));~~

```
constexpr iterator<false> end()
requires ((!simple-view<First> || ... || !simple-view<Vs>)
&& cartesian-product-is-common<First, Vs...>);
constexpr iterator<true> end() const
requires cartesian-product-is-common<const First, const Vs...>;
```

-4- Let:

(4.1) — *is-const* be true for the const-qualified overload_s, and false otherwise;

(4.2) — *is-end* be true for the end overloads, and false otherwise;

(4.2) — *is-empty* be true if the expression `ranges::empty(rng)` is true for any `rng` among the underlying ranges except the first one and false otherwise; and

(4.3) — *begin-or-first-end(rng)* be expression-equivalent to `is-end || is-empty ? cartesian-common-arg-end(rng) : ranges::begin(rng) is-empty ? ranges::begin(rng) : cartesian-common-arg-end(rng)` if `cartesian-product-common-arg<maybe-const<is-const, First>> is true and` `rng` is the first underlying range, and `ranges::begin(rng)` otherwise.

-5- Effects: Equivalent to:

```
iterator<is-const> it(tuple-transform(
    [](auto& rng){ return begin-or-first-end(rng); }, bases_));
return it;
```

2. Modify 25.7.33.3 [\[range.cartesian.iterator\]](#) as indicated:

```
friend constexpr bool operator==(const iterator& x, default_sentinel_t);
```

-26- Returns:

```
(?1) — If cartesian-product-common-arg<maybe-const<Const, First>> is true, returns std::get<0>(x.current) == ranges::end(std::get<0>(x.parent ->bases)).

(?2) — Otherwise, true if std::get<i>(x.current) == ranges::end(std::get<i>(x.parent ->bases)) is true for any integer  $0 \leq i \leq \text{sizeof} \dots (\text{Vs})$ ; otherwise, false returns true.

(?3) — Otherwise, returns false.
```

[2026-03-27; Tim provides new wording]

[Croydon 2026-03-27; Status changed: Open → Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Edit 24.3.4.7 [\[iterator.concept.sentinel\]](#), as indicated:

```
template<class S, class I>
concept sentinel_for =
    semiregular<S> &&
    input_or_output_iterator<I> &&
    weakly-equality-comparable-with<S, I>;    // see [concept.equalitycomparable]
```

-2- Let *s* and *i* be values of type *S* and *I* such that [*i*, *s*) denotes a range. Types *S* and *I* model `sentinel_for<S, I>` only if:

- (2.1) — *i* == *s* is well-defined.
- (2.2) — If `bool(i != s)` then *i* is dereferenceable and `[++i, s)` denotes a range.
- (2.3) — `assignable_from<I&, S>` is either modeled or not satisfied.
- (2.4) — If *I* and *S* are the same type, then *i* == *i* is true.

[3797](#). `elements_view` insufficiently constrained

Section: 25.7.23.2 [\[range.elements.view\]](#) Status: [Immediate](#) Submitter: Hui Xie Opened: 2022-10-21 Last modified: 2026-03-27

Priority: 2

View all other [issues](#) in `[range.elements.view]`.

Discussion:

This issue came up when I tried to integrate the C++23 changes to tuple-like into `ranges::elements_view` in `libc++`. Given the following test:

```
Using SubRange = ranges::subrange<MoveOnlyIter, Sent>;
std::vector<SubRange> srs = ...; // a vector of subranges
for(auto&& iter : srs | views::elements<0>){
}
```

The above code results in a hard error in deciding the `iterator_category` (The base is a random access range so it should exist). The immediate hard error complains that the following expression is invalid.

```
std::get<N>(*current_);
```

Note that even if `iterator_category` does not complain, it will complain later when we dereference the iterator.

Here are the declarations of the "get" overloads for `subrange`:

```
template<size_t N, class I, class S, subrange_kind K>
requires ((N == 0 && copyable<I>) || N == 1)
constexpr auto get(const subrange<I, S, K>& r);

template<size_t N, class I, class S, subrange_kind K>
requires (N < 2)
constexpr auto get(subrange<I, S, K>&& r);
```

Note that the first overload requires `copyable<I>` which is false and the second overload requires an rvalue, which is also not the case. So we don't have a valid "get" in this case.

But why does `elements_view` allow the instantiation in the first place? Let's look at its requirements:

```
template<class T, size_t N>
concept returnable_element =           // exposition only
    is_reference_v<T> || move_constructible<tuple_element_t<N, T>>;

template<input_range V, size_t N>
requires view<V> && has_tuple_element<range_value_t<V>, N> &&
    has_tuple_element<remove_reference_t<range_reference_t<V>>, N> &&
    returnable_element<range_reference_t<V>, N>
class elements_view;
```

It passed the "is_reference_v<range_reference_t<V>>" requirement, because it is "subrange&". Here the logic has an assumption: if the tuple-like is a reference, then we can always "get" and return a reference. This is not the case for subrange. subrange's get always return by value.

[2022-11-01; Reflector poll]

Set priority to 2 after reflector poll.

"The actual issue is that [P2165](#) broke *has-tuple-element* for this case. We should unbreak it."

Previous resolution [SUPERSEDED]:

This wording is relative to [N4917](#).

[Drafting Note: Three mutually exclusive options are prepared, depicted below by **Option A**, **Option B**, and **Option C**, respectively.]

Option A: Properly disallow this case (preferred solution)

1. Modify 25.7.23.2 [\[range.elements.view\]](#) as indicated:

```
namespace std::ranges {
    [...]
    template<class T, size_t N>
    concept returnable_element =                // exposition only
        requires { std::get<N>(declval<T>()); } &&
        is_reference_v<T> || move_constructible<tuple_element_t<N, T>>;
    [...]
}
```

Option B: Relax subrange's get to have more overloads. Since subrange's non-const begin unconditionally moves the iterator (even for lvalue-reference),

```
[[nodiscard]] constexpr I begin() requires (!copyable<I>);
Effects: Equivalent to: return std::move(begin_);
```

if we add more get overloads, it would work. The non-const lvalue-ref overload would work (and it also moves because non-const lvalue begin moves). This solution would make another way to let subrange's iterator in moved-from state, which is not good.

1. Modify 25.2 [\[ranges.syn\]](#) as indicated:

```
[...]
namespace std::ranges {
    [...]

    template<size_t N, class I, class S, subrange_kind K>
    requires ((N == 0 && copyable<I>) || N == 1)
    constexpr auto get(const subrange<I, S, K>& r);

    template<size_t N, class I, class S, subrange_kind K>
    requires (N < 2)
    constexpr auto get(subrange<I, S, K>&& r);

    template<size_t N, class I, class S, subrange_kind K>
    requires ((N == 0 && constructible_from<I, const I&&>) || N == 1)
    constexpr auto get(const subrange<I, S, K>&& r);

    template<size_t N, class I, class S, subrange_kind K>
    requires (N < 2)
    constexpr auto get(subrange<I, S, K>& r);
}
[...]
```

Option C: Make subrange's get to return by reference. This seems to significantly change the subrange's tuple protocol, which is not ideal.

[2026-03-27; Tim provides new wording]

[Croydon 2026-03-27; Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 25.7.23.2 [\[range.elements.view\]](#) as indicated:

```
namespace std::ranges {
    template<class T, size_t N>
    concept has_tuple_element =                // exposition only
        tuple_like<T> && N < tuple_size_v<T> &&
        requires(T t) {
            { std::get<N>(t) } -> convertible_to<const tuple_element_t<N, T>&>;
        };
}
```

```
    [...]
}
```

3891. LWG 3870 breaks `std::expected<cv T, E>`

Section: 22.8.6.1 [\[expected.object.general\]](#) Status: [Immediate](#) Submitter: Jiang An Opened: 2023-02-19 Last modified: 2026-03-27

Priority: 2

View all other [issues](#) in `[expected.object.general]`.

Discussion:

Currently the `value_type` of `std::expected` can be a *cv*-qualified type, which is possibly intended. However, LWG [3870](#)⁽¹⁾ disallows `std::construct_at` to construct objects via `cv T*`, which breaks `std::expected<cv T, E>` because some operations are specified with `std::construct_at` (22.8.6.4 [\[expected.object.assign\]](#), 22.8.6.5 [\[expected.object.swap\]](#)).

I think when `T` is *cv*-qualified, it would be better to store `std::remove_cv_t<T>` subobject while sometimes (other than construction/destruction) access it via a *cv*-qualified glvalue, which can also avoid UB associated with `const/volatile` objects.

[2023-03-22; Reflector poll]

Set priority to 2 after reflector poll. "Not clear if all these wording changes are needed or desired." "Unconvinced that the mixed-value-error swap should use `value()`, source is destroyed immediately anyway. The else branch should use `remove_cv_t` too."

Previous resolution [SUPERSEDED]:

This wording is relative to [N4928](#).

[Drafting note: When assignment and swap need to backup the old value by move construction, the source should be considered *cv*-unqualified, as the backup mechanism is only used internally.]

1. Modify 22.8.6.1 [\[expected.object.general\]](#) as indicated:

```
[...]
bool has_val;           // exposition only
union {
    remove_cv_t<T> val;   // exposition only
    E unex;               // exposition only
};
[...]
```

2. Modify 22.8.6.4 [\[expected.object.assign\]](#) as indicated:

```
constexpr expected& operator=(const expected& rhs);
```

-2- Effects:

(2.1) — If `this->has_value()` && `rhs.has_value()` is true, equivalent to `value().val = *rhs`.

[...]

[...]

```
constexpr expected& operator=(expected&& rhs) noexcept(see below);
```

[...]

-6- Effects:

(6.1) — If `this->has_value()` && `rhs.has_value()` is true, equivalent to `value().val = std::move(*rhs)`.

[...]

[...]

```
template<class U = T>
constexpr expected& operator=(U&& v);
```

[...]

-10- Effects:

(10.1) — If `has_value()` is true, equivalent to `value().val = std::forward<U>(v)`.

[...]

3. Modify Table 64: `swap(expected&)` effects [tab:expected.object.swap] as indicated:

Table 64 — swap(expected&) effects [tab:expected.object.swap]

	this->has_value()	!this->has_value()
rhs.has_value()	equivalent to: using std::swap; swap(value(), val, rhs.value(), val);	calls rhs.swap(*this)
	[...]	

4. Modify 22.8.6.5 [expected.object.swap] as indicated:

```
constexpr void swap(expected& rhs) noexcept(see below);
```

-1- *Constraints*: [...]

-2- *Effects*: See Table 64 [tab:expected.object.swap].

For the case where rhs.value() is false and this->has_value() is true, equivalent to:

```
if constexpr (is_nothrow_move_constructible_v<E>) {
    E tmp(std::move(rhs.unex));
    destroy_at(addressof(rhs.unex));
    try {
        construct_at(addressof(rhs.val), std::move(value(), val));
        destroy_at(addressof(val));
        construct_at(addressof(unex), std::move(tmp));
    } catch (...) {
        construct_at(addressof(rhs.unex), std::move(tmp));
        throw;
    }
} else {
    T tmp(std::move(val));
    destroy_at(addressof(val));
    try {
        construct_at(addressof(unex), std::move(rhs.unex));
        destroy_at(addressof(rhs.unex));
        construct_at(addressof(rhs.val), std::move(tmp));
    } catch (...) {
        construct_at(addressof(val), std::move(tmp));
        throw;
    }
}
has_val = false;
rhs.has_val = true;
```

[2024-10-02; Jonathan provides improved wording]

Removed the use of value() in the [expected.object.swap] p2 *Effects*: and added remove_cv_t to the local T in the else-branch.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4988](#).

[Drafting note: When assignment and swap need to backup the old value by move construction, the source should be considered cv-unqualified, as the backup mechanism is only used internally.]

1. Modify 22.8.6.1 [expected.object.general] as indicated:

```
[...]
bool has_val;          // exposition only
union {
    remove_cv_t<T> val;    // exposition only
    E unex;               // exposition only
};
[...]
```

2. Modify 22.8.6.4 [expected.object.assign] as indicated:

```
constexpr expected& operator=(const expected& rhs);
```

-2- *Effects*:

(2.1) — If this->has_value() && rhs.has_value() is true, equivalent to value(), val = *rhs.

[...]

[...]

```
constexpr expected& operator=(expected&& rhs) noexcept(see below);
```

[...]

-6- *Effects*:

(6.1) — If `this->has_value()` && `rhs.has_value()` is true, equivalent to `value().val = std::move(*rhs)`.

[...]

[...]

```
template<class U = T>
constexpr expected& operator=(U&& v);
```

[...]

-10- *Effects*:

(10.1) — If `has_value()` is true, equivalent to `value().val = std::forward<U>(v)`.

[...]

3. Modify Table 64: `swap(expected&)` effects [tab:expected.object.swap] as indicated:

Table 64 — `swap(expected&)` effects [tab:expected.object.swap]

	<code>this->has_value()</code>	<code>!this->has_value()</code>
<code>rhs.has_value()</code>	equivalent to: using <code>std::swap</code> ; <code>swap(value().val, rhs.value().val);</code>	calls <code>rhs.swap(*this)</code>
[...]		

4. Modify 22.8.6.5 [expected.object.swap] as indicated:

```
constexpr void swap(expected& rhs) noexcept(see below);
```

-1- *Constraints*: [...]

-2- *Effects*: See Table 64 [tab:expected.object.swap].

For the case where `rhs.value()` is false and `this->has_value()` is true, equivalent to:

```
if constexpr (is_nothrow_move_constructible_v<E>) {
    E tmp(std::move(rhs.unex));
    destroy_at(addressof(rhs.unex));
    try {
        construct_at(addressof(rhs.val), std::move(val));
        destroy_at(addressof(val));
        construct_at(addressof(unex), std::move(tmp));
    } catch (...) {
        construct_at(addressof(rhs.unex), std::move(tmp));
        throw;
    }
} else {
    remove_cv_t<T> tmp(std::move(val));
    destroy_at(addressof(val));
    try {
        construct_at(addressof(unex), std::move(rhs.unex));
        destroy_at(addressof(rhs.unex));
        construct_at(addressof(rhs.val), std::move(tmp));
    } catch (...) {
        construct_at(addressof(val), std::move(tmp));
        throw;
    }
}
has_val = false;
rhs.has_val = true;
```

[Croydon 2026-03-27; Jonathan provides new wording]

LWG review pointed out that using `value()` relies on a freestanding-deleted function. The equivalent functions in optional just use `val = rhs.val` and we can do the same here. We don't need to disallow assignment to `const T` here in the general case, because the assignments and swaps are already constrained to only work if the type is assignable/swappable. If your type satisfies both `assignable_from<const T&, const T&>` and `assignable_from<T&, const T&>` but with different semantics, this function might have surprising effects. That seems fine.

[Croydon 2026-03-27; Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

[Drafting note: When assignment and swap need to backup the old value by move construction, the source should be considered *cv-unqualified*, as the backup mechanism is only used internally.]

1. Modify 22.8.6.1 [expected.object.general] as indicated:

```
[...]
bool has_val;           // exposition only
```

```

union {
    remove_cv_t<T> val; // exposition only
    E unex; // exposition only
};
[...]
```

2. Modify 22.8.6.5 [\[expected.object.swap\]](#) as indicated:

```
constexpr void swap(expected& rhs) noexcept(see below);
```

-1- *Constraints*: [...]

-2- *Effects*: See Table 64 [tab:expected.object.swap].

For the case where `rhs.value()` is false and `this->has_value()` is true, equivalent to:

```

if constexpr (is_nothrow_move_constructible_v<E>) {
    E tmp(std::move(rhs.unex));
    destroy_at(addressof(rhs.unex));
    try {
        construct_at(addressof(rhs.val), std::move(val));
        destroy_at(addressof(val));
        construct_at(addressof(unex), std::move(tmp));
    } catch (...) {
        construct_at(addressof(rhs.unex), std::move(tmp));
        throw;
    }
} else {
    remove_cv_t<T> tmp(std::move(val));
    destroy_at(addressof(val));
    try {
        construct_at(addressof(unex), std::move(rhs.unex));
        destroy_at(addressof(rhs.unex));
        construct_at(addressof(rhs.val), std::move(tmp));
    } catch (...) {
        construct_at(addressof(val), std::move(tmp));
        throw;
    }
}
has_val = false;
rhs.has_val = true;
```

4026. Assignment operators of `std::expected` should propagate triviality

Section: 22.8.6.4 [\[expected.object.assign\]](#), 22.8.7.4 [\[expected.void.assign\]](#) **Status:** [Immediate](#) **Submitter:** Jiang An **Opened:** 2023-12-16 **Last modified:** 2026-03-25

Priority: 2

View all other [issues](#) in [expected.object.assign].

Duplicate of: [4195](#)

Discussion:

Addresses [US 135-216](#) and [US 136-217](#)

Currently, only copy and move constructors of `std::expected` are required to propagate triviality, while copy and move assignment operators are not. Given that the assignment operators of `std::optional` and `std::variant` are already required to propagate triviality, it seems to me that we should also apply such requirements for `std::expected`.

Such changes are being made in libc++ ([llvm/llvm-project#74768](#)). And it may be desired to make the triviality improvement portable.

[2024-03-11; Reflector poll]

Set priority to 2 after reflector poll in January 2024. A few votes for Tentatively Ready, others thought it needed more consideration.

[2026-01-29; added links to C++26 NB comments.]

[Croydon 2026-03-25; move to Immediate.]

Proposed resolution:

This wording is relative to [N4971](#).

1. Modify 22.8.6.4 [\[expected.object.assign\]](#) as indicated:

```
constexpr expected& operator=(const expected& rhs);
```

-2- *Effects*: [...]

[...]

-4- *Remarks*: This operator is defined as deleted unless:

[...]

-?- This operator is trivial if:

(2.1) — is trivially copy constructible $v<T>$ is true, and

(2.2) — is trivially copy assignable $v<T>$ is true, and

(2.3) — is trivially destructible $v<T>$ is true, and

(2.4) — is trivially copy constructible $v<E>$ is true, and

(2.5) — is trivially copy assignable $v<E>$ is true, and

(2.6) — is trivially destructible $v<E>$ is true.

constexpr expected& operator=(expected&& rhs) noexcept(see below);

-5- Constraints: [...]

[...]

-8- Remarks: The exception specification is equivalent to:

[...]

-?- This operator is trivial if:

(2.1) — is trivially move constructible $v<T>$ is true, and

(2.2) — is trivially move assignable $v<T>$ is true, and

(2.3) — is trivially destructible $v<T>$ is true, and

(2.4) — is trivially move constructible $v<E>$ is true, and

(2.5) — is trivially move assignable $v<E>$ is true, and

(2.6) — is trivially destructible $v<E>$ is true.

2. Modify 22.8.7.4 [\[expected.void.assign\]](#) as indicated:

constexpr expected& operator=(const expected& rhs);

-1- Effects: [...]

[...]

-3- Remarks: This operator is defined as deleted unless `is_copy_assignable_v<E>` is true and `is_copy_constructible_v<E>` is true.

-?- This operator is trivial if `is_trivially_copy_constructible_v<E>`, `is_trivially_copy_assignable_v<E>`, and `is_trivially_destructible_v<E>` are all true.

constexpr expected& operator=(expected&& rhs) noexcept(see below);

-4- Effects: [...]

[...]

-6- Remarks: The exception specification is equivalent to `is_nothrow_move_constructible_v<E>` && `is_nothrow_move_assignable_v<E>`.

-7- This operator is defined as deleted unless `is_move_constructible_v<E>` is true and `is_move_assignable_v<E>` is true.

-?- This operator is trivial if `is_trivially_move_constructible_v<E>`, `is_trivially_move_assignable_v<E>`, and `is_trivially_destructible_v<E>` are all true.

4122. Ill-formed operator<=> can cause hard error when instantiating std::inplace_vector

Section: 23.3.16.1 [\[inplace.vector.overview\]](#) Status: [Immediate](#) Submitter: Jiang An Opened: 2024-07-20 Last modified: 2026-03-27

Priority: 2

Discussion:

This is almost the same problem as LWG [4071^{\(f\)}](#) except that it happens to `std::inplace_vector`. As the `operator<=>` overload for `std::inplace_vector` is a non-template function whose return type (*synth-three-way-result* $<T>$) is not deduced, when the return type is ill-formed, hard error occurs in the instantiation of the enclosing `std::inplace_vector<T, N>`.

[2024-08-02; Reflector poll]

Set priority to 2 after reflector poll.

[Croydon 2026-03-27; Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to [N4986](#).

1. Modify 23.3.16.1 [\[inplace_vector.overview\]](#), class template `std::inplace_vector` synopsis, as indicated:

```
namespace std {
    template<class T, size_t N>
    class inplace_vector {
    public:
        [...]
        constexpr friend bool operator==(const inplace_vector& x,
                                         const inplace_vector& y);
        constexpr friend synth-three-way-result<T> auto
            operator<=>(const inplace_vector& x, const inplace_vector& y) +
            requires requires (const T t) { synth-three-way(t, t); }
        {
            return lexicographical_compare_three_way(x.begin(), x.end(), y.begin(), y.end(),
                synth-three-way);
        }
        constexpr friend void swap(inplace_vector& x, inplace_vector& y)
            noexcept(N == 0 || (is_nothrow_swappable_v<T> &&
                               is_nothrow_move_constructible_v<T>))
        { x.swap(y); }
    };
};
```

[4133](#). *awaitable-receiver*'s members are potentially throwing

Section: 33.13.1 [\[exec.as.awaitable\]](#) Status: [Immediate](#) Submitter: Eric Niebler Opened: 2024-07-30 Last modified: 2026-03-27

Priority: 1

View other [active issues](#) in [\[exec.as.awaitable\]](#).

View all other [issues](#) in [\[exec.as.awaitable\]](#).

Discussion:

The specification of *awaitable-receiver* in 33.13.1 [\[exec.as.awaitable\]](#)/p4 as of [N4988](#) is not taking into consideration the fact that the `resume()` and `promise()` member functions on `coroutine_handle<P>` are not marked `noexcept`. *awaitable-receiver*'s member functions must all be `noexcept`, but they are specified as being "equivalent to" statement that call `resume()` and `promise()` outside of `try/catch` blocks.

[2024-08-21; Reflector poll]

Set priority to 1 after reflector poll.

`promise()` can probably be *Throws: Nothing* (along with a bunch of other `coroutine_handle` members), but `resume()` certainly can throw. Also `AS-EXCEPT-PTR` can throw for the `error_code` case (that might be worth a separate issue though).

[2026-03-27; Tim adds wording following LWG discussion]

[Croydon 2026-03-27; Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 33.13.1 [\[exec.as.awaitable\]](#) as indicated:

-4- Let `rcvr` be an rvalue expression of type *awaitable-receiver*, let `crcvr` be a const lvalue that refers to `rcvr`, let `vs` be a pack of subexpressions, and let `err` be an expression of type `Err`. ~~Let `MAKE-NOEXCEPT(expr)` for some subexpression `expr` be expression-equivalent to `[&] noexcept -> decltype(auto) { return (expr); }()`.~~ Then:

(4.1) — If `constructible_from<result-type, decltype((vs))...>` is satisfied, the expression `set_value(rcvr, vs...)` is equivalent to:

```
try {
    rcvr.result_ptr->template emplace<1>(vs...);
} catch(...) {
    rcvr.result_ptr->template emplace<2>(current_exception());
}
MAKE-NOEXCEPT(rcvr.continuation.resume());
```

Otherwise, `set_value(rcvr, vs...)` is ill-formed.

(4.2) — The expression `set_error(rcvr, err)` is equivalent to:

```
try {
    rcvr.result_ptr->template emplace<2>(AS-EXCEPT-PTR(err));
```

```

} catch(...) {
    rcvr.result_ptr->template emplace<2>(current_exception());
}
MAKE_NOEXCEPT(rcvr.continuation.resume());

```

(4.3) — The expression `set_stopped(rcvr)` is equivalent to:

```
MAKE_NOEXCEPT(static_cast<coroutine_handle<>>(rcvr.continuation.promise()).unhandled_stopped()).resume();
```

(4.4) — For any expression `tag` whose type satisfies *forwarding-query* and for any pack of subexpressions `as`, `get_env(rcvr).query(tag, as...)` is expression-equivalent to:

```
tag(get_env(as_const(MAKE_NOEXCEPT(rcvr.continuation.promise()))), as...)
```

4143. `execution::set_value/set_error/set_stopped/start` should always return `void`

Section: 33.7.2 [\[exec.set.value\]](#), 33.7.3 [\[exec.set.error\]](#), 33.7.4 [\[exec.set.stopped\]](#), 33.8.2 [\[exec.opstate.start\]](#) **Status:** [Immediate](#) **Submitter:** Jiang An **Opened:** 2024-08-20 **Last modified:** 2026-03-27

Priority: 2

Discussion:

In editorial issue [#7222](#), it was observed that currently `execution::start` may have a non-void return value, which possibly interacts with overloaded operator,. But the return value of `execution::start` doesn't seem used anywhere.

In addition to `execution::start`, the return values of `execution::set_value`, `execution::set_error`, and `execution::set_stopped` also seem never used, and the return type of these CPOs are always `void` in [stdexec](#). Perhaps it would be better to specified in the standard that these CPOs always return `void`.

[2024-09-18; Reflector poll]

Set priority to 2 after reflector poll. Should require the expressions to have type `void`, rather than just discarding anything that is returned.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4988](#).

1. Modify 33.7.2 [\[exec.set.value\]](#) as indicated:

-1- `set_value` is a value completion function (33.3 [\[exec.async.ops\]](#)). Its associated completion tag is `set_value_t`. The expression `set_value(rcvr, vs...)` for a subexpression `rcvr` and pack of subexpressions `vs` is ill-formed if `rcvr` is an lvalue or an rvalue of const type. Otherwise, it is expression-equivalent to `MANDATE-NO_THROW(void(rcvr.set_value(vs...)))`.

2. Modify 33.7.3 [\[exec.set.error\]](#) as indicated:

-1- `set_error` is an error completion function (33.3 [\[exec.async.ops\]](#)). Its associated completion tag is `set_error_t`. The expression `set_error(rcvr, err)` for some subexpressions `rcvr` and `err` is ill-formed if `rcvr` is an lvalue or an rvalue of const type. Otherwise, it is expression-equivalent to `MANDATE-NO_THROW(void(rcvr.set_error(err)))`.

3. Modify 33.7.4 [\[exec.set.stopped\]](#) as indicated:

-1- `set_stopped` is a stopped completion function (33.3 [\[exec.async.ops\]](#)). Its associated completion tag is `set_stopped_t`. The expression `set_stopped(rcvr)` for a subexpression `rcvr` is ill-formed if `rcvr` is an lvalue or an rvalue of const type. Otherwise, it is expression-equivalent to `MANDATE-NO_THROW(void(rcvr.set_stopped()))`.

4. Modify 33.8.2 [\[exec.opstate.start\]](#) as indicated:

-1- The name `start` denotes a customization point object that starts (33.3 [\[exec.async.ops\]](#)) the asynchronous operation associated with the operation state object. For a subexpression `op`, the expression `start(op)` is ill-formed if `op` is an rvalue. Otherwise, it is expression-equivalent to `MANDATE-NO_THROW(void(op.start()))`.

[2026-03-27; Tim provides new wording]

[Croydon 2026-03-27; Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to [N4988](#).

1. Modify 33.7.2 [\[exec.set.value\]](#) as indicated:

-1- `set_value` is a value completion function (33.3 [\[exec.async.ops\]](#)). Its associated completion tag is `set_value_t`. The expression `set_value(rcvr, vs...)` for a subexpression `rcvr` and pack of subexpressions `vs` is ill-formed if `rcvr` is an lvalue or an rvalue of const type. Otherwise, it is expression-equivalent to `MANDATE-NO_THROW(rcvr.set_value(vs...))`.

Mandates: If the expression above is well-formed, its type is `void`.

2. Modify 33.7.3 [\[exec.set.error\]](#) as indicated:

-1- `set_error` is an error completion function (33.3 [\[exec.async.ops\]](#)). Its associated completion tag is `set_error_t`. The expression `set_error(rcvr, err)` for some subexpressions `rcvr` and `err` is ill-formed if `rcvr` is an lvalue or an rvalue of const type. Otherwise, it is expression-equivalent to `MANDATE-NO_THROW(rcvr.set_error(err))`.

Mandates: If the expression above is well-formed, its type is void.

3. Modify 33.7.4 [\[exec.set.stopped\]](#) as indicated:

-1- `set_stopped` is a stopped completion function (33.3 [\[exec.async.ops\]](#)). Its associated completion tag is `set_stopped_t`. The expression `set_stopped(rcvr)` for a subexpression `rcvr` is ill-formed if `rcvr` is an lvalue or an rvalue of const type. Otherwise, it is expression-equivalent to `MANDATE-NO_THROW(rcvr.set_stopped())`.

Mandates: If the expression above is well-formed, its type is void.

4. Modify 33.8.2 [\[exec.opstate.start\]](#) as indicated:

-1- The name `start` denotes a customization point object that starts (33.3 [\[exec.async.ops\]](#)) the asynchronous operation associated with the operation state object. For a subexpression `op`, the expression `start(op)` is ill-formed if `op` is an rvalue. Otherwise, it is expression-equivalent to `MANDATE-NO_THROW(op.start())`.

Mandates: If the expression above is well-formed, its type is void.

4151. Precondition of `inplace_vector::swap`

Section: 23.3.16.5 [\[inplace.vector.modifiers\]](#) **Status:** [Immediate](#) **Submitter:** Arthur O'Dwyer **Opened:** 2024-09-07 **Last modified:** 2026-03-27

Priority: 2

Discussion:

Right now `inplace_vector::swap` has only a declaration in the class synopsis; it doesn't specify what the behavior of `swap` for `inplace_vectors` actually is. So I think the behavior ends up being governed by 23.2.2.2 [\[container.reqmts\]](#), which are written from the point of view of a container that manages an external heap allocation so that swapping containers doesn't touch the elements at all, just changes their ownership.

`inplace_vector::swap` actually works more like `array::swap`, which has its own specification in 23.3.3.3 [\[array.members\]](#), where it is defined in terms of `std::swap_ranges`. The `std::swap_ranges` algorithm (26.7.3 [\[alg.swap\]](#)) has a precondition! This precondition is missing from `inplace_vector::swap`.

That is, I think we currently have no wording that explains why

```
std::pmr::monotonic_buffer_resource mr1;
std::inplace_vector<std::pmr::vector<int>, 2> v1, v2;
v1.emplace_back(1, &mr1);
v2.emplace_back(1);
v1.swap(v2);
```

is undefined behavior. The current spec seems to say this Just Works, even though it physically cannot work because `v1[0]` and `v2[0]` don't dynamically meet the semantic requirements of being "swappable with" each other, and `v1.swap(v2)` is necessarily going to try to swap them.

[2024-09-18; Reflector poll]

Set priority to 2 after reflector poll. The *Preconditions*: (but not the *Effects*;) would be changed again if [P3160R2](#) is approved.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4988](#).

1. Modify 23.3.16.5 [\[inplace.vector.modifiers\]](#) as indicated:

[Drafting note: It is suggested to add the new wording to the end of the existing subclause]

constexpr void swap(inplace_vector& x) noexcept(N == 0 ||
(is_nothrow_swappable_v<T> && is_nothrow_move_constructible_v<T>));

Preconditions: Let M be $\min(\text{size}(), x.\text{size}())$. For each non-negative integer $n < M$, $(*this)[n]$ is swappable with $(16.4.4.3 \text{ [swappable.requirements]}) x[n]$.

Effects: Exchanges the contents of $*this$ and x .

[Croydon 2026-03-27; Jonathan provides new wording]

Also require `Cpp17MoveConstructible`.

[Croydon 2026-03-27; Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 23.3.16.5 [\[inplace.vector.modifiers\]](#) as indicated:

[Drafting note: It is suggested to add the new wording to the end of the existing subclause]

```
constexpr void swap(inplace vector& x) noexcept(N == 0 ||
(is nothrow swappable v<T> && is nothrow move constructible v<T>));
```

Preconditions: T meets the *Cpp17MoveConstructible* requirements. Let M be $\min(\text{size}(), x.\text{size}())$. For each non-negative integer $n < M$, $(*this)[n]$ is swappable with $x[n]$ (16.4.4.3 [\[swappable.requirements\]](#)).

Effects: Exchanges the contents of $*this$ and x .

4223. Deduction guides for maps are mishandling tuples and references

Section: 23.4.1 [\[associative.general\]](#) Status: [Immediate](#) Submitter: Tomasz Kaminski Opened: 2025-03-14 Last modified: 2026-03-27

Priority: 2

Discussion:

The `from_range` deduction guide for maps currently do not handle ranges of tuple of two elements:

```
std::vector<int> v;
auto zv = std::views::zip(v, v);
std::map m4(std::from_range, zv); // Ill-formed, no-deduction guide
```

This seems to be result of merge conflict between [P2165](#) (Compatibility between tuple, pair and tuple-like objects) and [P1206R4](#) (Conversions from ranges to containers): The helper *range-key-type* and *range-mapped-type* aliases introduced by the later use the old formulation of `::first_type`, `::second_type` instead of `tuple_element`.

Furthermore, both iterator and range deduction guides do not correctly handle iterators with a pair of references as value types, and deduce key or value type as reference, which is ill-formed:

```
std::flat_map<int, float> fm; // iterator value_type is pair<int, float>
std::map m1(fm.begin(), fm.end()); // OK, deduces std::map<int, float>

auto tv = fm | std::views::transform(std::identity{}); // iterator value value_type is pair<int const&, float const&>
std::map m3(tv.begin(), tv.end()); // Ill-formed, deduces std::map<int const&, float&>
```

[2025-08-29; Reflector poll]

Set priority to 2 after reflector poll.

[Croydon 2026-03-27; Status changed: New $\rightarrow$ Immediate.]

Proposed resolution:

This wording is relative to [N5001](#).

[Drafting note: The proposed change also strips `const` from the value type of the map, changing the behavior of previously working code:

```
std::pair<int const, float const> tp[2];
std::map m(std::begin(tp), std::end(tp)); // Was std::map<int, float const>, now std::map<int, float>

]
```

1. Modify 23.4.1 [\[associative.general\]](#) as indicated:

```
template<class InputIterator>
using iter-value-type =
    typename iterator_traits<InputIterator>::value_type; // exposition only

template<class InputIterator>
using iter-key-type = remove_const_t remove_cvref_t<
    tuple_element_t<0, iter-value-type<InputIterator>>>; // exposition only

template<class InputIterator>
using iter-mapped-type = remove_cvref_t<
    tuple_element_t<1, iter-value-type<InputIterator>>>; // exposition only

template<class InputIterator>
using iter-to-alloc-type = pair<
    add_const_t<
        tuple_element_t<0, iter-value-type<InputIterator>>>
        iter-key-type<InputIterator>
    >,
    tuple_element_t<1, iter-value-type<InputIterator>>>
    iter-mapped-type<InputIterator>
>; // exposition only

template<ranges::input_range Range>
using range-key-type =
```



```

remove_const_t<typename ranges::range_value_t<Range>::first_type>
remove_cvref_t<tuple element t<0, ranges::range_value_t<Range>>>; // exposition only

template<ranges::input_range Range>
using range-mapped-type =
    typename ranges::range_value_t<Range>::second_type
remove_cvref_t<tuple element t<1, ranges::range_value_t<Range>>>; // exposition only

template<ranges::input_range Range>
using range-to-alloc-type =
    pair<add_const_t<
        typename ranges::range_value_t<Range>::first_type
        range-key-type<Range>
    >,
        typename ranges::range_value_t<Range>::second_type
        range-mapped-type<Range>
    >; // exposition only

```

4314. Missing move in mdspan layout mapping::operator()

Section: 23.7.3.4 [[mdspan.layout](#)] **Status:** [Immediate](#) **Submitter:** Luc Grosheintz **Opened:** 2025-08-13 **Last modified:** 2026-03-25

Priority: 2

View all other [issues](#) in [[mdspan.layout](#)].

Discussion:

Numerous template classes in <mdspan> have template parameter `IndexType`. While this template parameter is restricted to be a signed or unsigned integer, these classes often accept user-defined classes that convert to `IndexType`.

They're either passed as an array/span of `OtherIndexType`; or as a template parameter pack. When passed as a template parameter pack, the common pattern is

```

template<class... OtherIndexTypes>
requires std::is_convertible_v<OtherIndexTypes, IndexType> && ...
void dummy(OtherIndexTypes... indices)
{
    something(static_cast<IndexType>(std::move(indices))...);
}

```

This pattern allows passing in objects that convert to `IndexType` only as an rvalue reference, e.g.

```

class RValueInt
{
    constexpr
    operator int() && noexcept
    { return m_int; }

private:
    int m_int;
};

```

This pattern can be found in:

- a ctor of `extents`,
- a ctor of `mdspan`,
- `mdspan::operator[]`.

The five standardized layout mappings use a different pattern in their `operator()`, namely,

```
static_cast<IndexType>(indices)...
```

This prevents the passing in objects of type `RValueInt`, because the conversion isn't happening from an rvalue reference. This is addressed by Items 1 - 5 in the Proposed Resolution.

A different pattern can be found a ctor for `layout_{left,right}_padded`. Namely, directly passing an object of type `OtherIndexType` to *LEAST-MULTIPLE-AT-LEAST*. This is addressed in Items 6 and 7 in the Proposed Resolution.

This inconsistency was noticed while fixing [PR121061](#) and these changes have been applied to all layout mappings implemented in `libstdc++`.

[2025-10-21; Reflector poll.]

Set priority to 2 after reflector poll.

The resolution bypasses `extents_type::index-cast` that would validate if input value is representable.

We should require convertibility without regard to `const` and value category.

Previous resolution [SUPERSEDED]:

This wording is relative to [N5014](#).

1. Modify 23.7.3.4.5.3 [\[mdspan.layout.left.obs\]](#) as indicated:

```
template<class... Indices>
constexpr index_type operator()(Indices... i) const noexcept;

-2- Constraints: [...]
```

-3- Preconditions: [...]

-4- Effects: Let P be a parameter pack such that

$$\text{is_same_v}<\text{index_sequence_for}<\text{Indices...}>, \text{index_sequence}<\text{P...}>>$$

is true. Equivalent to:

$$\text{return } ((\text{static_cast}<\text{index_type}>(\text{std::move}(i)) * \text{stride}(P)) + \dots + 0);$$

2. Modify 23.7.3.4.6.3 [\[mdspan.layout.right.obs\]](#) as indicated:

```
template<class... Indices>
constexpr index_type operator()(Indices... i) const noexcept;

-2- Constraints: [...]
```

-3- Preconditions: [...]

-4- Effects: Let P be a parameter pack such that

$$\text{is_same_v}<\text{index_sequence_for}<\text{Indices...}>, \text{index_sequence}<\text{P...}>>$$

is true. Equivalent to:

$$\text{return } ((\text{static_cast}<\text{index_type}>(\text{std::move}(i)) * \text{stride}(P)) + \dots + 0);$$

3. Modify 23.7.3.4.7.4 [\[mdspan.layout.stride.obs\]](#) as indicated:

```
template<class... Indices>
constexpr index_type operator()(Indices... i) const noexcept;

-2- Constraints: [...]
```

-3- Preconditions: [...]

-4- Effects: Let P be a parameter pack such that

$$\text{is_same_v}<\text{index_sequence_for}<\text{Indices...}>, \text{index_sequence}<\text{P...}>>$$

is true. Equivalent to:

$$\text{return } ((\text{static_cast}<\text{index_type}>(\text{std::move}(i)) * \text{stride}(P)) + \dots + 0);$$

4. Modify 23.7.3.4.8.4 [\[mdspan.layout.leftpad.obs\]](#) as indicated:

```
template<class... Indices>
constexpr index_type operator()(Indices... idxs) const noexcept;

-3- Constraints: [...]
```

-4- Preconditions: [...]

-5- Returns: $((\text{static_cast}<\text{index_type}>(\text{std::move}(\text{idxs})) * \text{stride}(P_rank)) + \dots + 0).$

5. Modify 23.7.3.4.9.4 [\[mdspan.layout.rightpad.obs\]](#) as indicated:

```
template<class... Indices>
constexpr index_type operator()(Indices... idxs) const noexcept;

-3- Constraints: [...]
```

-4- Preconditions: [...]

-5- Returns: $((\text{static_cast}<\text{index_type}>(\text{std::move}(\text{idxs})) * \text{stride}(P_rank)) + \dots + 0).$

6. Modify 23.7.3.4.8.3 [\[mdspan.layout.leftpad.cons\]](#) as indicated:

```
template<class OtherIndexType>
constexpr mapping(const extents_type& ext, OtherIndexType padding);

Let pad = static_cast<index_type>(std::move(padding)).
```

-3- Constraints: [...]

-4- Preconditions:

(4.1) — padding is representable as a value of type index_type.

(4.2) — ~~extents_type::index-cast(pad)~~pad is greater than zero.

(4.3) — If *rank_* is greater than one, then *LEAST-MULTIPLE-AT-LEAST*(pad, ext.extent(0)) is representable as a value of type *index_type*.

(4.4) — If *rank_* is greater than one, then the product of *LEAST-MULTIPLE-AT-LEAST*(pad, ext.extent(0)) and all values ext.extent(*k*) with *k* in the range of [1, *rank_*) is representable as a value of type *index_type*.

(4.5) — If padding_value is not equal to dynamic_extent, padding_value equals ~~extents_type::index-cast(pad)~~pad.

-5- Effects: [...]

7. Modify 23.7.3.4.9.3 [mdspan.layout.rightpad.cons] as indicated:

```
template<class OtherIndexType>
constexpr mapping(const extents_type& ext, OtherIndexType padding);
```

Let pad = static_cast<index_type>(std::move(padding)).

-3- Constraints: [...]

-4- Preconditions:

(4.1) — padding is representable as a value of type *index_type*.

(4.2) — ~~extents_type::index-cast(pad)~~pad is greater than zero.

(4.3) — If *rank_* is greater than one, then *LEAST-MULTIPLE-AT-LEAST*(pad, ext.extent(*rank_* - 1)) is representable as a value of type *index_type*.

(4.4) — If *rank_* is greater than one, then the product of *LEAST-MULTIPLE-AT-LEAST*(pad, ext.extent(*rank_* - 1)) and all values ext.extent(*k*) with *k* in the range of [1, *rank_* - 1) is representable as a value of type *index_type*.

(4.5) — If padding_value is not equal to dynamic_extent, padding_value equals ~~extents_type::index-cast(pad)~~pad.

-5- Effects: [...]

[2025-10-05; Tomasz provides upated wording after LWG review]

[Croydon 2026-03-25; move to Immediate.]

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 23.7.3.3.2 [mdspan.extents.expo] as indicated:

```
template<class OtherIndexType>
static constexpr auto index-cast(OtherIndexType&& i) noexcept;
```

-9- Effects:

- o -9.1- If OtherIndexType is an integral type other than bool, then equivalent to return i;
- o -9.2- otherwise, equivalent to return static_cast<index_type>(std::forward<OtherIndexType>(i)).

2. Modify 23.7.3.4.5.3 [mdspan.layout.left.obs] as indicated:

```
template<class... Indices>
constexpr index_type operator()(Indices... i) const noexcept;
```

-2- Constraints: [...]

-3- Preconditions: [...]

-4- Effects: Let P be a parameter pack such that

is_same_v<index_sequence_for<Indices...>, index_sequence<P...>>

is true. Equivalent to:

return ((static_cast<index_type>(std::move(i)) * stride(P)) + ... + 0);

3. Modify 23.7.3.4.6.3 [mdspan.layout.right.obs] as indicated:

```
template<class... Indices>
constexpr index_type operator()(Indices... i) const noexcept;
```

-2- Constraints: [...]

-3- *Preconditions*: [...]

-4- *Effects*: Let P be a parameter pack such that

`is_same_v<index_sequence_for<Indices...>, index_sequence<P...>>`

is true. Equivalent to:

`return ((static_cast<index_type>(std::move(i)) * stride(P)) + ... + 0);`

4. Modify 23.7.3.4.7.4 [\[mdspan.layout.stride.obs\]](#) as indicated:

```
template<class... Indices>
constexpr index_type operator()(Indices... i) const noexcept;
```

-2- *Constraints*: [...]

-3- *Preconditions*: [...]

-4- *Effects*: Let P be a parameter pack such that

`is_same_v<index_sequence_for<Indices...>, index_sequence<P...>>`

is true. Equivalent to:

`return ((static_cast<index_type>(std::move(i)) * stride(P)) + ... + 0);`

5. Modify 23.7.3.4.8.4 [\[mdspan.layout.leftpad.obs\]](#) as indicated:

```
template<class... Indices>
constexpr index_type operator()(Indices... idxs) const noexcept;
```

-3- *Constraints*: [...]

-4- *Preconditions*: [...]

-5- *Returns*: `((static_cast<index_type>(std::move(idxs)) * stride(P_rank)) + ... + 0).`

6. Modify 23.7.3.4.9.4 [\[mdspan.layout.rightpad.obs\]](#) as indicated:

```
template<class... Indices>
constexpr index_type operator()(Indices... idxs) const noexcept;
```

-3- *Constraints*: [...]

-4- *Preconditions*: [...]

-5- *Returns*: `((static_cast<index_type>(std::move(idxs)) * stride(P_rank)) + ... + 0).`

7. Modify 23.7.3.4.8.3 [\[mdspan.layout.leftpad.cons\]](#) as indicated:

```
template<class OtherIndexType>
constexpr mapping(const extents_type& ext, OtherIndexType padding);
```

Let pad be `extents_type::index-cast(std::move(padding))`.

-3- *Constraints*: [...]

-4- *Preconditions*:

(4.1) — pad is representable as a value of type `index_type`.

(4.2) — ~~`extents_type::index-cast(pad)`~~ pad is greater than zero.

(4.3) — If `rank_` is greater than one, then `LEAST-MULTIPLE-AT-LEAST(pad, ext.extent(0))` is representable as a value of type `index_type`.

(4.4) — If `rank_` is greater than one, then the product of `LEAST-MULTIPLE-AT-LEAST(pad, ext.extent(0))` and all values `ext.extent(k)` with `k` in the range of `[1, rank_)` is representable as a value of type `index_type`.

(4.5) — If `padding_value` is not equal to `dynamic_extent`, `padding_value` equals ~~`extents_type::index-cast(pad)`~~ pad.

-5- *Effects*: Direct-non-list-initializes `extents_` with `ext`, and if `rank_` is greater than one, direct-non-list-initializes `stride-rm2` with `LEAST-MULTIPLE-AT-LEAST(pad, ext.extent(rank_ - 1))`.

8. Modify 23.7.3.4.9.3 [\[mdspan.layout.rightpad.cons\]](#) as indicated:

```
template<class OtherIndexType>
constexpr mapping(const extents_type& ext, OtherIndexType padding);
```

Let pad be `extents_type::index-cast(std::move(padding))`.

-3- *Constraints*: [...]

-4- Preconditions:

- (4.1) — `pad` is representable as a value of type `index_type`.
- (4.2) — `extents_type::index_cast(pad)` is greater than zero.
- (4.3) — If `rank_` is greater than one, then `LEAST-MULTIPLE-AT-LEAST(pad, ext.extent(rank_ - 1))` is representable as a value of type `index_type`.
- (4.4) — If `rank_` is greater than one, then the product of `LEAST-MULTIPLE-AT-LEAST(pad, ext.extent(rank_ - 1))` and all values `ext.extent(k)` with `k` in the range of `[1, rank_ - 1]` is representable as a value of type `index_type`.
- (4.5) — If `padding_value` is not equal to `dynamic_extent`, `padding_value` equals `extents_type::index_cast(pad)`.

-5- Effects: Direct-non-list-initializes `extents_` with `ext`, and if `rank_` is greater than one, direct-non-list-initializes `stride-rm2` with `LEAST-MULTIPLE-AT-LEAST(pad, ext.extent(rank_ - 1))`.

4325. `std::indirect`'s `operator==` still does not support incomplete types

Section: 20.4.1.8 [indirect.relops], 20.4.1.9 [indirect.comp.with.t] Status: [Immediate](#) Submitter: Hewill Kang Opened: 2025-08-24 Last modified: 2026-03-27

Priority: 2

Discussion:

`std::indirect`'s `operator==` [intentionally](#) uses *Mandates* instead of *Constraints* to support incomplete types. However, its function signature has the following `noexcept` specification:

```
template<class U, class AA>
constexpr bool operator==(const indirect& lhs, const indirect<U, AA>& rhs)
    noexcept(noexcept(*lhs == *rhs));
```

That is, we check whether the expression `*lhs == *rhs` throws, which unfortunately leads to the following hard error:

```
struct Incomplete;
static_assert(std::equality_comparable<std::indirect<Incomplete>>);
// hard error, no match for 'operator==' (operand types are 'const Incomplete' and 'const Incomplete')
```

This makes `operator==` not SFINAE-friendly for incomplete types, which defeats the purpose.

Also, checking `noexcept(*lhs == *rhs)` seems insufficient because the result of `*lhs == *rhs` might still throw during conversion to `bool`.

We could add a note similar to 22.10.6.1 [refwrap.general].

[2025-10-14; Reflector poll]

Set priority to 2 after reflector poll.

Even if we don't want to support incomplete types, we still need to fix the `noexcept`-specifier to consider whether the conversion to `bool` throws, e.g. `noexcept(noexcept(bool(*lhs == *rhs)))`.

"The proposed resolution may result in the `noexcept` operator giving different results when evaluated in different translation units where the type `T` of `indirect` was incomplete or not. Ill-formed seems safer than inconsistent."

Previous resolution [SUPERSEDED]:

This wording is relative to [N5014](#).

[Drafting note:: We introduce the exposition-only function *FUN* below to mimic the implicit conversion to `bool`. As a drive-by effect this helps us simplifying (and clarifying, see LWG [484\(ii\)](#)) the existing *Mandates* element.

Please note that the seemingly unresolved `T` in the requires expression below names the first template parameter of the `indirect` class template.]

1. Modify 20.4.1.8 [indirect.relops] as indicated:

```
template<class U, class AA>
constexpr bool operator==(const indirect& lhs, const indirect<U, AA>& rhs)
    noexcept(noexcept(*lhs == *rhs) see below);
```

.-? Let FUN denote the exposition-only function

bool FUN(bool) noexcept;

-1- *Mandates*: The expression *FUN(*lhs == *rhs)* is well-formed *and its result is convertible to bool*.

-2- *Returns*: If `lhs` is valueless or `rhs` is valueless, `lhs.valueless_after_move() == rhs.valueless_after_move()`; otherwise `*lhs == *rhs`.

-?- Remarks: The exception specification is equivalent to:

```
requires (const T& lhs, const U& rhs) { { FUN(lhs == rhs) } noexcept; }
```

2. Modify 20.4.1.9 [\[indirect.comp.with.t\]](#) as indicated:

```
template<class U>
constexpr bool operator==(const indirect& lhs, const U& rhs) noexcept(noexcept(*lhs == rhs)see below);
```

-?- Let *FUN* denote the exposition-only function

```
bool FUN(bool) noexcept;
```

-1- Mandates: The expression *FUN*(*lhs == rhs) is well-formed ~~and its result is convertible to bool~~.

-2- Returns: If lhs is valueless, false; otherwise *lhs == rhs.

-?- Remarks: The exception specification is equivalent to:

```
requires (const T& lhs, const U& rhs) { { FUN(lhs == rhs) } noexcept; }
```

[2026-03-26; Tim provides new wording]

This only fixes the exception specification to include conversion to bool.

[Croydon 2026-03-27; Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 20.4.1.8 [\[indirect.relops\]](#) as indicated:

```
template<class U, class AA>
constexpr bool operator==(const indirect& lhs, const indirect<U, AA>& rhs)
noexcept(noexcept(bool(*lhs == *rhs)));
```

-1- [...]

2. Modify 20.4.1.9 [\[indirect.comp.with.t\]](#) as indicated:

```
template<class U>
constexpr bool operator==(const indirect& lhs, const U& rhs) noexcept(noexcept(bool(*lhs == rhs)));
```

-1- [...]

[4339](#). task's coroutine frame may be released late

Section: 33.13.6 [\[exec.task\]](#) **Status:** [Immediate](#) **Submitter:** Dietmar Kühl **Opened:** 2025-08-31 **Last modified:** 2026-03-27

Priority: 2

Discussion:

Addresses [US 242-372](#)

The specification of task doesn't spell out when the coroutine frame is destroyed (i.e., when `handle.destroy()` is called). As a result the lifetime of arguments passed to the coroutine and/or of local variables in the coroutine body may be longer than expected.

The intention is that the coroutine frame is destroyed before any of the completion functions is called. One implication of this requirement is that the result and error objects can't be stored in the `promise_type` when the completion function is called although the exposition-only members *result* and *errors* imply exactly that. Instead the data needs to be stored in the operation state object or it needs to be moved to a different place before calling `destroy()`.

The proposed resolution is to add a paragraph to the specification of `promise_type` in 33.13.6.5 [\[task.promise\]](#) that spells out that the coroutine frame is destroyed before any of the completion functions is called. To actually do that the exposition-only members *result* and *errors* can't remain as members of the `promise_type`. While writing the relevant change it turned out that *errors* is a variant which only ever stores an `exception_ptr` (the other potential errors are immediately reported via the awaiter return from `yield_value`). Thus, the variant can be replaced with an `exception_ptr`.

[2025-10-17; Reflector poll.]

Set priority to 2 after reflector poll.

"P/R is incomplete - also needs to update at least `promise_type::uncaught_exception()`."

[Kona 2025-11-06]

SG1 think resolution looks good, but noted pre-existing issue that 7.3 doesn't use `std::move` on `*st.result`. (Would need to be `*std::move(st.result)` to handle optional<T&> correctly.)

LWG [4415](#)⁽¹⁾ renames `uncaught_exception` to `unhandled_exception`.

Previous resolution [SUPERSEDED]:

In 33.13.6.4 [task.state], add exposition-only data members *result* and *error* to the exposition-only class *state*:

```
namespace std::execution {
    template<class T, class Environment>
    template<receiver Rcvr>
    class task<T, Environment>::state {           // exposition only
    ...
        Environment          environment; // exposition only
        optional<T>          result;      // exposition only; present only if is_void v<T> is false
        exception_ptr        error;      // exposition-only
    };
}
```

Remove the exposition-only data members *result* and *errors* from the class *promise_type* in 33.13.6.5 [task.promise]:

```
namespace std::execution {
    template<class T, class Environment>
    class task<T, Environment>::promise_type {
    ...
        stop_token_type token; // exposition only
        optional<T> result; // exposition only; present only if is_void v<T> is false
        error_variant errors; // exposition only
    };
}
```

The definition of *error-variant* isn't needed, i.e., remove 33.13.6.5 [task.promise] paragraph 2:

~~2- *error-variant* is a variant<monostate, remove_cvref_t<E>...>, with duplicate types removed, where E... are template arguments of the specialization of *execution::completion_signatures* denoted by *error_types*.~~

In 33.13.6.5 [task.promise] change paragraph 7 to use the members added to *state*:

auto final_suspend() noexcept

-7- *Returns*: An awaitable object of unspecified type (7.6.2.4 [expr.await]) whose member functions arrange for the completion of the asynchronous operation associated with *STATE(*this)* ~~by invoking~~. Let *st* be a reference to *STATE(*this)*. The asynchronous completion first destroys the coroutine frame using *st.handle.destroy()* and then invokes:

- -7.1- ~~set_error(std::move(RCVR(*this).rcvr), std::move(~~est.error~~))~~ if ~~errors.index()~~ is greater than zero and ~~e is the value held by errors~~ bool(st.error) is true, otherwise
- -7.2- ~~set_value(std::move(RCVR(*this).rcvr))~~ if *is_void<T>* is true, and otherwise
- -7.3- ~~set_value(std::move(RCVR(*this).rcvr), *st.result).~~

Change the specification of *yield_value* to destroy the coroutine frame before invoking the *set_error* completion, i.e., change 33.13.6.5 [task.promise] paragraph 9:

-9- *Returns*: An awaitable object of unspecified type ([expr.await]) whose member functions arrange for the calling coroutine to be suspended and then completes the asynchronous operation associated with *STATE(*this)* ~~by~~. Let *st* be a reference to *STATE(*this)*. Then the asynchronous operation completes by first destroying the coroutine frame using *st.handle.destroy()* and then invoking *set_error*(std::move(RCVR(*this).rcvr), Cerr(std::move(err.error))).

Change the specification of *unhandled_stopped* to destroy the coroutine frame before invoking the *set_stopped* completion, i.e., change 33.13.6.5 [task.promise] paragraph 13:

coroutine_handle<> unhandled_stopped();

-13- *Effects*: Completes the asynchronous operation associated with *STATE(*this)* ~~by~~. Let *st* be a reference to *STATE(*this)*. The asynchronous operation is completed by first destroying the coroutine frame using *st.handle.destroy()* and then invoking *set_stopped*(std::move(RCVR(*this).rcvr)).

[2025-11-07; Jonathan provides improved wording]

Incorporate change from LWG 4466⁽ⁱ⁾ and add *_v* to *is_void*.

LWG moved this to Ready but then reopened it because it still needs an update w.r.t *unhandled_exception*.

[2026-01-16; LWG telecon. Dietmar will add a paragraph changing how *unhandled_exception* reports errors.]

Previous resolution [SUPERSEDED]:

In 33.13.6.4 [task.state], add exposition-only data members *result* and *error* to the exposition-only class *state*:

```
namespace std::execution {
    template<class T, class Environment>
    template<receiver Rcvr>
    class task<T, Environment>::state {           // exposition only
    ...
        Environment          environment; // exposition only
        optional<T>          result;      // exposition only; present only if is_void v<T> is false
        exception_ptr        error;      // exposition-only
    };
}
```

```
};
}
```

Remove the exposition-only data members *result* and *errors* from the class *promise_type* in 33.13.6.5 [task.promise]:

```
namespace std::execution {
    template<class T, class Environment>
    class task<T, Environment>::promise_type {
    ...
    stop_token_type token; // exposition only
    optional<T> result; // exposition only; present only if is_void_v<T> is false
    error_variant errors; // exposition only
    };
}
```

The definition of *error-variant* isn't needed, i.e., remove 33.13.6.5 [task.promise] paragraph 2:

~~2- *error-variant* is a variant<monostate, remove_cvref_t<E>...>, with duplicate types removed, where E... are template arguments of the specialization of *execution::completion_signatures* denoted by *error_types*.~~

In 33.13.6.5 [task.promise] change paragraph 7 to use the members added to *state*:

```
auto final_suspend() noexcept
```

-7- *Returns*: An awaitable object of unspecified type (7.6.2.4 [expr.await]) whose member functions arrange for the completion of the asynchronous operation associated with *STATE(*this)* by invoking. Let *st* be a reference to *STATE(*this)*. The asynchronous completion first destroys the coroutine frame using *st.handle.destroy()* and then invokes:

- -7.1- *set_error*(std::move(RCVR(*this).st.rcvr), std::move(est.error)) if *errors.index()* is greater than zero and *e* is the value held by *errors*. *bool(st.error)* is true, otherwise
- -7.2- *set_value*(std::move(RCVR(*this).st.rcvr)) if *is_void_v<T>* is true, and otherwise
- -7.3- *set_value*(std::move(RCVR(*this).st.rcvr), *std::move(st.result)).

Change the specification of *yield_value* to destroy the coroutine frame before invoking the *set_error* completion, i.e., change 33.13.6.5 [task.promise] paragraph 9:

-9- *Returns*: An awaitable object of unspecified type ([expr.await]) whose member functions arrange for the calling coroutine to be suspended and then completes the asynchronous operation associated with *STATE(*this)* by. Let *st* be a reference to *STATE(*this)*. Then the asynchronous operation completes by first destroying the coroutine frame using *st.handle.destroy()* and then invoking *set_error*(std::move(RCVR(*this).st.rcvr), Cerr(std::move(err.error))).

Change the specification of *unhandled_stopped* to destroy the coroutine frame before invoking the *set_stopped* completion, i.e., change 33.13.6.5 [task.promise] paragraph 13:

```
coroutine_handle<> unhandled_stopped();
```

-13- *Effects*: Completes the asynchronous operation associated with *STATE(*this)* by. Let *st* be a reference to *STATE(*this)*. The asynchronous operation is completed by first destroying the coroutine frame using *st.handle.destroy()* and then invoking *set_stopped*(std::move(RCVR(*this).st.rcvr)).

[2026-01-22; Dietmar Kühl provides improved wording]

[Croydon 2026-03-27; move to Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. In 33.13.6.4 [task.state], add exposition-only data members *result* and *error* to the exposition-only class *state*:

```
namespace std::execution {
    template<class T, class Environment>
    template<receiver Rcvr>
    class task<T, Environment>::state { // exposition only
    ...
    Environment environment; // exposition only
    optional<T> result; // exposition only; present only if is_void_v<T> is false
    exception_ptr error; // exposition-only
    };
}
```

2. Remove the exposition-only data members *result* and *errors* from the class *promise_type* in 33.13.6.5 [task.promise]:

```
namespace std::execution {
    template<class T, class Environment>
    class task<T, Environment>::promise_type {
    ...
    stop_token_type token; // exposition only
    optional<T> result; // exposition only; present only if is_void_v<T> is false
    error_variant errors; // exposition only
    };
}
```

3. The definition of *error-variant* isn't needed, i.e., remove 33.13.6.5 [task.promise] paragraph 2:

~~2- error_variant is a variant<monostate, remove_cvref_t<E>...>, with duplicate types removed, where E... are template arguments of the specialization of execution::completion_signatures denoted by error_types.~~

4. In 33.13.6.5 [\[task.promise\]](#) change paragraph 6 to use the members added to `state`:

```
auto final_suspend() noexcept
```

-6- *Returns*: An awaitable object of unspecified type (7.6.2.4 [\[expr.await\]](#)) whose member functions arrange for the completion of the asynchronous operation associated with `STATE(*this)` ~~by invoking~~. **Let `st` be a reference to `STATE(*this)`. The asynchronous completion first destroys the coroutine frame using `st.handle.destroy()` and then invokes:**

(6.1) — `set_error(std::move(RCVR(*this).st.rcvr), std::move(st.error))` if ~~`errors.index()` is greater than zero~~ **and `e` is the value held by `errors` `bool(st.error)` is true**, otherwise
(6.2) — `set_value(std::move(RCVR(*this).st.rcvr))` if `is_void_v<T>` is true, and otherwise
(6.3) — `set_value(std::move(RCVR(*this).st.rcvr), *std::move(st.result))`.

5. In 33.13.6.5 [\[task.promise\]](#) change the specification of `yield_value` to destroy the coroutine frame before invoking the `set_error` completion:

-8- *Returns*: An awaitable object of unspecified type (7.6.2.4 [\[expr.await\]](#)) whose member functions arrange for the calling coroutine to be suspended and then completes the asynchronous operation associated with `STATE(*this)` ~~by~~. **Let `st` be a reference to `STATE(*this)`. Then the asynchronous operation completes by first destroying the coroutine frame using `st.handle.destroy()` and then** invoking `set_error(std::move(RCVR(*this).st.rcvr), Cerr(std::move(err.error)))`.

6. In 33.13.6.5 [\[task.promise\]](#) change the specification of `unhandled_exception` to use the `error` member of `STATE(*this)`:

```
void unhandled_exception();
```

-11- *Effects*: If the signature `set_error_t(exception_ptr)` is not an element of `error_types`, calls `terminate()` (14.6.2 [\[except.terminate\]](#)). Otherwise, stores `current_exception()` into `STATE(*this).error` ~~`errors`~~.

7. In 33.13.6.5 [\[task.promise\]](#) change the specification of `unhandled_stopped` to destroy the coroutine frame before invoking the `set_stopped` completion:

```
coroutine_handle<> unhandled_stopped();
```

-12- *Effects*: Completes the asynchronous operation associated with `STATE(*this)` ~~by~~. **Let `st` be a reference to `STATE(*this)`. The asynchronous operation is completed by first destroying the coroutine frame using `st.handle.destroy()` and then** invoking `set_stopped(std::move(RCVR(*this).st.rcvr))`.

4347. task's stop source is always created

Section: 33.13.6.5 [\[task.promise\]](#) Status: [Immediate](#) Submitter: Dietmar Kühl Opened: 2025-09-01 Last modified: 2026-03-27

Priority: 2

View other [active issues](#) in [\[task.promise\]](#).

View all other [issues](#) in [\[task.promise\]](#).

Discussion:

Addresses [US 257-382](#)

The type `task<...>::promise_type` has exposition-only members `source` and `token`. These can be interpreted as always existing which would be a performance issue for former and an unnecessary constraints for the latter (because stop tokens aren't required to be default constructible).

The intent is that the stop token obtained from the `get_stop_token` query of the receiver's environment is used. Only if this type is different from the task's `stop_token_type` a stop source of type `stop_source_type` needs to be created when the `get_stop_token` query is used on the promise type's environment. The stop token doesn't need to be stored at all: it can either be obtained from the receiver's environment or from the stop source. The fix is to show the stop source as an optionally present member of of the operation state and it should be of type `std::optional<stop_source_type>` to imply that it is only created when accessed.

[2025-10-17; Reflector poll.]

Set priority to 2 after reflector poll.

[Croydon 2026-03-27; Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Change the synopsis in 33.13.6.4 [\[task.state\]](#) to include an `optional<stop_source_type>` member and an exposition-only function to get a stop token:

```
namespace std::execution {
    template<class T, class Environment>
    template<receiver Rcvr>
    class task<T, Environment>::state {          // exposition only
    public:
        using operation_state_concept = operation_state_t;

        template<class R>
        state(coroutine_handle<promise_type> h, R&& rr);
```

```

~state();

void start() & noexcept;

stop_token_type get-stop-token(); // exposition only

private:
using own-env-t = see below; // exposition only
coroutine_handle<promise_type> handle; // exposition only
remove_cvref_t<Rcvr> rcvr; // exposition only
optional<stop_source_type> source; // exposition only
own-env-t own-env; // exposition only
Environment environment; // exposition only
};
}

```

2. In 33.13.6.4 [\[task.state\]](#) change paragraph 4 into two paragraphs: one for `start()` and one for `get-stop-token()` such that the initialization of `source` is in the second paragraph:

```
void start() & noexcept;
```

-4- *Effects:* Let `prom` be the object `handle.promise()`. Associates `STATE(prom)`, `RCVR(prom)`, and `SCHED(prom)` with `*this` as follows:

-4.1- -- `STATE(prom)` is `*this`.

-4.2- -- `RCVR(prom)` is `rcvr`.

-4.3- -- `SCHED(prom)` is the object initialized with `scheduler_type(get_scheduler(get_env(rcvr)))` if that expression is valid and `scheduler_type()` otherwise. If neither of these expressions is valid, the program is ill-formed.

~~Let `st` be `get-stop-token(get_env(rcvr))`. Initializes `prom.token` and `prom.source` such that~~

~~-4.4- -- `prom.token.stop_requested()` returns `st.stop_requested()`;~~

~~-4.5- -- `prom.token.stop_possible()` returns `st.stop_possible()`; and~~

~~-4.6- -- for types `Fn` and `Init` such that both `invocable<Fn>` and `constructible_from<Fn, Init>` are modeled, `stop_token_type::callback_type<Fn>` models `stoppable_callback_for<Fn, stop_token_type, Init>`;~~

-5- After that invokes `handle.resume()`.

`stop_token_type get-stop-token();`

-6- *Effects:* If `same_as<decltype(declval<stop_source_type>().get_token()), decltype(get-stop-token(get_env(rcvr)))>` is true returns `get-stop-token(get_env(rcvr))`. Otherwise, if `source.has_value()` is false, initializes the contained value of `source` such that

- o -6.1- -- `source->stop_requested()` returns `get-stop-token(get_env(rcvr))->stop_requested()`; and
- o -6.2- -- `source->stop_possible()` returns `get-stop-token(get_env(rcvr))->stop_possible()`

Finally, returns `source->get_token()`.

3. Remove `token` and `source` from the synopsis of `promise_type` in 33.13.6.5 [\[task.promise\]](#):

```

namespace std::execution {
template<class T, class Environment>
class task<T, Environment>::promise_type {
public:
...

private:
using error-variant = see below; // exposition only

allocator_type alloc; // exposition only
stop_source_type source; // exposition only
stop_token_type token; // exposition only
optional<T> result; // exposition only; present only if is_void_v<T> is false
error-variant errors; // exposition only
};
}

```

4. Change the specification of `get_env` in 33.13.6.5 [\[task.promise\]](#) paragraph 16.3 to use `STATE(*this).get-stop-token()`:

unspecified `get_env()` const noexcept;

-16- *Returns:* An object `env` such that queries are forwarded as follows:

-16.1- -- `env.query(get_scheduler)` returns `scheduler_type(SCHED(*this))`.

-16.2- -- `env.query(get_allocator)` returns `alloc`.

-16.3- -- `env.query(get_stop_token)` returns `tokenSTATE(*this).get-stop-token()`.

-16.4- -- For any other query `q` and arguments `a...` a call to `env.query(q, a...)` returns `STATE(*this).environmentenvironment.query(q, a...)` if this expression is well-formed and `forwarding_query(q)` is well-formed and is true. Otherwise `env.query(q, a...)` is ill-formed.

4354. Reconsider weakly_parallel as the default forward_progress_guarantee

Section: 33.5.8 [\[exec.get.fwd.progress\]](#) Status: [Immediate](#) Submitter: Lewis Baker Opened: 2025-08-25 Last modified: 2026-03-27

Priority: 1

Discussion:

The `get_forward_progress_guarantee` CPO is intended to allow querying a scheduler as for what sort of forward-progress guarantee it provides. Algorithms may use this to determine whether it is safe to execute certain operations on a given scheduler. If a scheduler does not customize this query, the query will fall back to returning a forward-progress guarantee of `weakly_parallel`.

I think we should reconsider what this default should be returning and suggest it should instead return `parallel` by default, as this will be by far the most common kind of scheduler, i.e. a scheduler that executes on `std::thread`-like execution agents and that maintains a queue of scheduled tasks.

I expect it to be common that authors of schedulers may forget to customize the `get_forward_progress_guarantee_t` query and just leave it at the default. This will likely leave their scheduler reporting a weaker guarantee than it actually provides and thus not being usable within generic algorithms that require at least `parallel` forward progress.

For example, the `run_loop` execution context defined in 33.12.1 [\[exec.run.loop\]](#) does not define its scheduler to customize the `get_forward_progress_guarantee_t`. This means it will report the default value of `weakly_parallel`.

However, the scheduled operations will run on the thread that calls `run_loop::run()` and thus will inherit its forward-progress guarantees. As this function might block and is therefore unsafe to invoke it from a thread/agent with `weakly_parallel` forward progress guarantees (which should probably be explicitly specified as having undefined-behaviour) we can safely assume that `run_loop`'s scheduler can provide `parallel` forward-progress guarantee.

It's not clear whether the current `run_loop` specification defaulting to its scheduler having `weakly_parallel` forward progress guarantee is intentional or unintentional here. However, forgetting to define the `get_forward_progress_guarantee` query on a scheduler is something I expect to be fairly common.

Schedulers that provide `weakly_parallel` (or in future, `concurrent`) forward progress guarantees require implementations to be much more aware of the fact that these are the guarantees they are providing and thus could be more expected to customize the `get_forward_progress_guarantee` query to return the respective values.

[2025-10-20; Reflector poll. Status changed: New → SG1]

Set priority to 1 after reflector poll. Send to SG1 and LEWG.

"If there is a default, it should be the weakest possible one. If that is an unfortunate choice I'd rather prefer no default and mandate that the query gets implemented. Providing a default which is stronger than the weakest possible creates logic errors. Accidentally claiming weaker than the actual value is only a performance error."

"This is tension between the default being promising the least and the default being the most likely thing a user wants to do. Assuming the least powerful guarantees unless the user has opted in is safer. Changing this choice requires going back to LEWG or SG1."

"Plenty of reasonable schedulers are weakly parallel at best. It's the right default. If your scheduler offers better than that, you would naturally remember to customize it."

"It seems that the authors of `run_loop::scheduler` did not naturally remember to customize it. It's possible the intent was that `run_loop::scheduler` should not offer better than `weakly_parallel` forward progress, but it was not discussed in P2300. The absence of an explicit implementation of the query could either be intentional or an accidental omission. Perhaps this is an indication that there should not be a default forward-progress guarantee for schedulers?"

[Kona 2025-11-06; SG1 feedback.]

SG1 recommends to remove the default `get_forward_progress_guarantee` implementation instead of adopting LWG4354 current proposed resolution and encourages a paper that explores that direction.

Previous resolution [SUPERSEDED]:

This wording is relative to [N5014](#).

1. Modify 33.5.8 [\[exec.get.fwd.progress\]](#) as indicated:

-2- The name `get_forward_progress_guarantee` denotes a query object. For a subexpression `sch`, let `Sch` be `decltype((sch))`. If `Sch` does not satisfy `scheduler`, `get_forward_progress_guarantee` is ill-formed. Otherwise, `get_forward_progress_guarantee(sch)` is expression-equivalent to:

(2.1) — `MANDATE-NO_THROW(AS-CONST(sch).query(get_forward_progress_guarantee))` if that expression is well-formed.

Mandates: The type of the expression above is `forward_progress_guarantee`.

(2.2) — Otherwise, `forward_progress_guarantee::weakly_parallel`.

[2026-03-26; Jonathan provides new wording]

Previous resolution [SUPERSEDED]:

This wording is relative to [N5032](#).

1. Modify 33.5.8 [\[exec.get.fwd.progress\]](#) as indicated:

-2- The name `get_forward_progress_guarantee` denotes a query object. For a subexpression `sch`, let `Sch` be `decltype((sch))`. If `Sch` does not satisfy `scheduler`, `get_forward_progress_guarantee` is ill-formed. Otherwise, `get_forward_progress_guarantee(sch)` is expression-equivalent to:

~~(2.1) — MANDATE-NO_THROW(AS-CONST(sch).query(get_forward_progress_guarantee)) if that expression is well-formed.~~

Mandates: The type of the expression above is `forward_progress_guarantee`.

~~(2.2) — Otherwise, `forward_progress_guarantee::weakly_parallel`.~~

[2026-03-26; Ruslan provides new wording]

[Croydon 2026-03-27; move to Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 33.5.8 [\[exec.get.fwd.progress\]](#) as indicated:

-2- The name `get_forward_progress_guarantee` denotes a query object. For a subexpression `sch`, let `Sch` be `decltype((sch))`. If `Sch` does not satisfy `scheduler`, `get_forward_progress_guarantee` is ill-formed. Otherwise, `get_forward_progress_guarantee(sch)` is expression-equivalent to:

~~(2.1) — MANDATE-NO_THROW(AS-CONST(sch).query(get_forward_progress_guarantee)) if that expression is well-formed.~~

Mandates: The type of the expression above is `forward_progress_guarantee`.

~~(2.2) — Otherwise, `forward_progress_guarantee::weakly_parallel`.~~

2. Modify 33.6 [\[exec.sched\]](#) as indicated:

```
template<class Sch>
concept scheduler =
    derived_from<typename remove_cvref_t<Sch>::scheduler_concept, scheduler_t> &&
    queryable<Sch> &&
    requires(Sch&& sch) {
        { schedule(std::forward<Sch>(sch)) } -> sender;
        get_forward_progress_guarantee(sch) -> same_as<forward_progress_guarantee>;
        { auto(get_completion_scheduler<set_value_t>(
            get_env(schedule(std::forward<Sch>(sch)))) )
            -> same_as<remove_cvref_t<Sch>>;
        } &&
        equality_comparable<remove_cvref_t<Sch>> &&
        copyable<remove_cvref_t<Sch>>;
```

[4361](#). `awaitable-receiver::set_value` should use *Mandates* instead of constraints

Section: 33.13.1 [\[exec.as.awaitable\]](#) Status: [Immediate](#) Submitter: Lewis Baker Opened: 2025-08-28 Last modified: 2026-03-27

Priority: 1

View other [active issues](#) in [\[exec.as.awaitable\]](#).

View all other [issues](#) in [\[exec.as.awaitable\]](#).

Discussion:

In 33.13.1 [\[exec.as.awaitable\]](#) bullet 4.1 the `awaitable-receiver::set_value` member function is defined as having a constraint that the *result-type* is constructible from the values.

If `constructible_from<result-type, decltype((vs))...>` is satisfied, the expression `set_value(rcvr, vs...)` is equivalent to:

```
try {
    rcvr.result_ptr->template emplace<1>(vs...);
} catch(...) {
    rcvr.result_ptr->template emplace<2>(current_exception());
}
rcvr.continuation.resume();
```

Otherwise, `set_value(rcvr, vs...)` is ill-formed.

Should we be using mandates here instead of constraints (or alternatively just drop the constraint altogether)? There shouldn't be any need to change behaviour based on whether or not the receiver's completion methods are well-formed or not.

It is worth noting that there is inconsistent use of constraints on `set_value` methods in other receiver implementations throughout 33 [\[exec\]](#).

For example: The following `set_value` member function applies constraints:

- In 33.9.2 [\[exec.snd.expos\]](#) `basic-receiver::set_value` constrains that check that it can accept those specific value arguments

While the following `set_value` member functions do not apply constraints:

- In 33.9.12.10 [\[exec.let\]](#) `receiver2::set_value`
- In 33.9.12.18 [\[exec.spawn.future\]](#) `spawn-future-receiver::set_value`
- in 33.9.13.1 [\[exec.sync.wait\]](#) `sync-wait-receiver::set_value`

We should probably try to be consistent on whether or not `set_value` implementations should use constraints or mandates. Given that it is not allowed to form calls to the receiver unless that overload is present in the `completion_signatures`, it may be worth just making them all mandates. This would tend to make uses of the `receiver_of` concept less useful as satisfying `receiver_of<R, Sig>` would not necessarily guarantee that actually trying to call each of R's corresponding completion functions will result in a well-formed program. It is arguable that this is already the status-quo, however.

[2025-10-23; Reflector poll. Status changed: New → LEWG]

Set priority to 1 after reflector poll.

"Send to LEWG if we want to go this way, which makes `receiver_of` mostly meaningless and calls into question why we even have the concept."

[Croydon 2026-03-27; LEWG confirmed the direction, [P4159R0](#) removes the concept.]

[Croydon 2026-03-27; Status changed: LEWG → Immediate.]

Proposed resolution:

This wording is relative to [N5014](#).

1. Modify 33.13.1 [\[exec.as.awaitable\]](#) as indicated:

-4. Let `rcvr` be an rvalue expression of type `awaitable-receiver`, let `crcvr` be a const lvalue that refers to `rcvr`, let `vs` be a pack of subexpressions, and let `err` be an expression of type `Err`. Then:

(4.1) — ~~If constructible from `result-type`, `decltype((vs))...` is satisfied, `t`~~ The expression `set_value(rcvr, vs...)` is equivalent to:

```
try {
    rcvr.result_ptr->template emplace<1>(vs...);
} catch(...) {
    rcvr.result_ptr->template emplace<2>(current_exception());
}
rcvr.continuation.resume();
```

~~Otherwise, `set_value(rcvr, vs...)` is ill-formed~~ Mandates: constructible from `result-type`, `decltype((vs))...` is satisfied.

(4.2) — [...]

(4.3) — [...]

(4.4) — [...]

2. Modify 33.9.2 [\[exec.snd.expos\]](#) after p25 as indicated:

```
[...]
template<class Sndr, class Rcvr, class Index>
requires valid-specialization<env-type, Index, Sndr, Rcvr>
struct basic-receiver { // exposition only
    using receiver_t;

    using tag_t = tag_of_t<Sndr>; // exposition only
    using state_t = state_type<Sndr, Rcvr>; // exposition only
    static constexpr const auto& complete = impls-for<tag_t>::complete; // exposition only

    template<class... Args>
    requires callable<decltype(complete), Index, state_t&, Rcvr&, set_value_t, Args...>
    void set_value(Args&&... args) && noexcept {
        complete(Index(), op->state, op->rcvr, set_value_t(), std::forward<Args>(args)...);
    }

    template<class Error>
    requires callable<decltype(complete), Index, state_t&, Rcvr&, set_error_t, Error>
    void set_error(Error&& err) && noexcept {
        complete(Index(), op->state, op->rcvr, set_error_t(), std::forward<Error>(err));
    }

    void set_stopped() && noexcept
    requires callable<decltype(complete), Index, state_t&, Rcvr&, set_stopped_t> {
        complete(Index(), op->state, op->rcvr, set_stopped_t());
    }

    auto get_env() const noexcept -> env_type<Index, Sndr, Rcvr> {
        return impls-for<tag_t>::get-env(Index(), op->state, op->rcvr);
    }

    basic-state<Sndr, Rcvr>* op; // exposition only
```

```
};  
[...]
```

[4379](#). `hive::reserve()` needs *Throws*: element adjusted to match block min/max considerations

Section: 23.3.9.3 [[hive.capacity](#)] **Status:** [Immediate](#) **Submitter:** Matt Bentley **Opened:** 2025-09-17 **Last modified:** 2026-03-25

Priority: 3

View other [active issues](#) in [[hive.capacity](#)].

View all other [issues](#) in [[hive.capacity](#)].

Discussion:

Addresses [US 142-236](#).

This issue comes from Bloomberg as part of their C++26 comments via Incits. To summarize their case, in a call to `reserve(n)`,

```
if (n > capacity() && capacity() + current-limits.min > max_size()),
```

`reserve` should throw, e.g. when `max_size=100`, `capacity=80`, `current-limits.min` and `current-limits.max` are 40 and `n=90`.

In addition, in the above scenario if we increase `max_size()` to 140 and `n` to 130, we can see that although we could add one block with a capacity of `current-limits.min`, adding another would be impossible; we still cannot make `capacity >= n` without also being `> max_size`.

This is currently not stated in the *Throws*: element. I've implemented the requested additional throws and they are easily achievable.

[2025-10-22; Reflector poll.]

Set priority to 3 after reflector poll.

There was discussion if `reserve()` is allowed to deallocate unused blocks, that materialized into LWG [4380^{\(1\)}](#).

Previous resolution [SUPERSEDED]:

This wording is relative to [N5014](#).

1. Modify 23.3.9.3 [[hive.capacity](#)] as indicated:

```
void reserve(size_type n);
```

-3- *Effects*: If `n <= capacity()` is true, there are no effects. Otherwise increases `capacity()` by allocating reserved blocks.

-4- *Postconditions*: `capacity() >= n` is true.

-5- *Throws*: `length_error` if `capacity() cannot be made >= n without being > max_size()`, as well as any exceptions thrown by the allocator.

```
[...]
```

[2026-03-24; Matt provides new wording]

[Croydon 2026-03-25; move to Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 23.3.9.3 [[hive.capacity](#)] as indicated:

```
void reserve(size_type n);
```

-3- *Effects*: If `n <= capacity()` is true, there are no effects. Otherwise increases `capacity()` by allocating reserved blocks.

-4- *Postconditions*: `capacity() >= n` is true.

-5- *Throws*: `length_error` if `n` $\rightarrow$ `satisfying the postcondition would cause capacity() to exceed max_size()`, as well as any exceptions thrown by the allocator.

```
[...]
```

[4476](#). `run_loop` should not have a `set_error` completion

Section: 33.12.1.1 [[exec.run.loop.general](#)] **Status:** [Immediate](#) **Submitter:** Eric Niebler **Opened:** 2025-11-16 **Last modified:** 2026-03-27

Priority: 2

Discussion:

When `run_loop` was proposed, the only implementation we had synchronized with a mutex and a condition variable. Operations on those can theoretically throw exceptions, so `run_loop` got a `set_error_t(exception_ptr)` completion signature.

Since then, [a lock-free implementation of `run\_loop` has been found](#). Atomic operations cannot fail with an exception, so an atomic `run_loop` can never complete with an error.

[2026-01-16; Reflector poll.]

Set priority to 2 after reflector poll.

"Very helpful to know if something completes without exceptions, it affects whether completions on some types of scheduler are correct."

"Want confirmation from LEWG as `run()` and `finish()` have narrow contracts."

[2026-03-26; Status New → LEWG.]

[Croydon 2026-03-27; LEWG confirmed they want `noexcept`. Status changes: LEWG → Open.]

Previous resolution [SUPERSEDED]:

This wording is relative to [N5014](#).

[Drafting note: It should be pointed out that member function `finish` should be `noexcept` for other reasons as well, see LWG [4215^{\(1\)}](#).]

1. Modify 33.12.1.1 [\[`exec.run.loop.general`\]](#), class `run_loop` synopsis, as indicated:

```
namespace std::execution {
  class run_loop {
    // 33.12.1.2 \[exec.run.loop.types\], associated types
    class run-loop-scheduler;           // exposition only
    class run-loop-sender;             // exposition only
    struct run-loop-opstate-base {      // exposition only
      virtual void execute() noexcept = 0; // exposition only
      run_loop* loop;                  // exposition only
      run-loop-opstate-base* next;      // exposition only
    };
    template<class Rcvr>
      using run-loop-opstate = unspecified; // exposition only

    // 33.12.1.4 \[exec.run.loop.members\], member functions
    run-loop-opstate-base* pop-front() noexcept; // exposition only
    void push-back(run-loop-opstate-base*) noexcept; // exposition only

  public:
    // 33.12.1.3 \[exec.run.loop.ctor\], constructor and destructor
    run_loop() noexcept;
    run_loop(run_loop&&) = delete;
    ~run_loop();

    // 33.12.1.4 \[exec.run.loop.members\] member functions
    run-loop-scheduler get_scheduler() noexcept;
    void run() noexcept;
    void finish() noexcept;
  };
}
```

2. Modify 33.12.1.2 [\[`exec.run.loop.types`\]](#) as indicated:

```
class run-loop-sender;
```

-5- `run-loop-sender` is an exposition-only type that satisfies `sender.completion_signatures_of_t<runloop-sender>` is

```
completion_signatures<set_value_t(), set_error_t(exception_ptr), set_stopped_t(>
```

[...]

-9- Let o be a non-const lvalue of type `run-loop-opstate<Rcvr>`, and let $\text{REC}(o)$ be a non-const lvalue reference to an instance of type `Rcvr` that was initialized with the expression `rcvr` passed to the invocation of `connect` that returned o . Then:

(9.1) — [...]

(9.2) — [...]

(9.3) — The expression `start(o)` is equivalent to:

```
try {
  o.loop->push-back(addressof(o));
} catch(...) {
  set_error(std::move(REC(o)), current_exception());
}
```

3. Modify 33.12.1.4 [\[`exec.run.loop.members`\]](#) as indicated:


```
run-loop-opstate-base* pop-front() noexcept;
```

-1- *Effects*: Blocks (3.6 [defns.block]) until one of the following conditions is true: [...]

```
void push-back(run-loop-opstate-base* item) noexcept;
```

-2- *Effects*: Adds *item* to the back of the queue and increments *count* by 1.

-3- *Synchronization*: This operation synchronizes with the *pop-front* operation that obtains *item*.

```
run-loop-scheduler get_scheduler() noexcept;
```

-4- *Returns*: An instance of *run-loop-scheduler* that can be used to schedule work onto this *run_loop* instance.

```
void run() noexcept;
```

-5- *Preconditions*: is either *starting* or *finishing*.

-6- *Effects*: If *state* is *starting*, sets the *state* to *running*, otherwise leaves *state* unchanged. Then, equivalent to:

```
while (auto* op = pop-front()) {
    op->execute();
}
```

-7- *Remarks*: When *state* changes, it does so without introducing data races.

```
void finish() noexcept;
```

-8- *Preconditions*: *state* is either *starting* or *running*.

-9- *Effects*: Changes *state* to *finishing*.

-10- *Synchronization*: *finish* synchronizes with the *pop-front* operation that returns *nullptr*.

[2026-03-27; Jonathan provides new wording]

Change to [exec.run.loop.types]/5 superseded by P3941.

[Croydon 2026-03-27; Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to N5014.

[Drafting note: It should be pointed out that member function *finish* should be *noexcept* for other reasons as well, see LWG 4215⁽¹⁾.]

1. Modify 33.12.1.1 [exec.run.loop.general], class *run_loop* synopsis, as indicated:

```
namespace std::execution {
    class run_loop {
        // 33.12.1.2 [exec.run.loop.types], associated types
        class run-loop-scheduler; // exposition only
        class run-loop-sender; // exposition only
        struct run-loop-opstate-base { // exposition only
            virtual void execute() noexcept = 0; // exposition only
            run_loop* loop; // exposition only
            run-loop-opstate-base* next; // exposition only
        };
        template<class Rcvr>
            using run-loop-opstate = unspecified; // exposition only

        // 33.12.1.4 [exec.run.loop.members], member functions
        run-loop-opstate-base* pop-front() noexcept; // exposition only
        void push-back(run-loop-opstate-base*) noexcept; // exposition only

    public:
        // 33.12.1.3 [exec.run.loop.ctor], constructor and destructor
        run_loop() noexcept;
        run_loop(run_loop&&) = delete;
        ~run_loop();

        // 33.12.1.4 [exec.run.loop.members] member functions
        run-loop-scheduler get_scheduler() noexcept;
        void run() noexcept;
        void finish() noexcept;
    };
}
```

2. Modify 33.12.1.2 [exec.run.loop.types] as indicated:

-9- Let *o* be a non-const lvalue of type *run-loop-opstate<Rcvr>*, and let *REC(o)* be a non-const lvalue reference to an instance of type *Rcvr* that was initialized with the expression *rcvr* passed to the invocation of *connect* that returned *o*. Then:

- (9.1) — [...]
- (9.2) — [...]
- (9.3) — The expression `start(o)` is equivalent to:

```
try {
    o->loop->push-back(addressof(o));
} catch(...) {
    set_error(std::move(REC(o)), current_exception());
}
```

3. Modify 33.12.1.4 [\[exec.run.loop.members\]](#) as indicated:

`run-loop-opstate-base* pop-front()` **noexcept**;

- 1- *Effects*: Blocks (3.6 [\[defs.block\]](#)) until one of the following conditions is true: [...]

`void push-back(run-loop-opstate-base* item)` **noexcept**;

- 2- *Effects*: Adds item to the back of the queue and increments *count* by 1.
- 3- *Synchronization*: This operation synchronizes with the *pop-front* operation that obtains item.

`run-loop-scheduler get_scheduler()` **noexcept**;

- 4- *Returns*: An instance of *run-loop-scheduler* that can be used to schedule work onto this *run_loop* instance.

`void run()` **noexcept**;

- 5- *Preconditions*: is either *starting* or *finishing*.
- 6- *Effects*: If *state* is *starting*, sets the *state* to *running*, otherwise leaves *state* unchanged. Then, equivalent to:

```
while (auto* op = pop-front()) {
    op->execute();
}
```

- 7- *Remarks*: When *state* changes, it does so without introducing data races.

`void finish()` **noexcept**;

- 8- *Preconditions*: *state* is either *starting* or *running*.
- 9- *Effects*: Changes *state* to *finishing*.
- 10- *Synchronization*: *finish* synchronizes with the *pop-front* operation that returns `nullptr`.

4478. meta::has_identifier is not specified for annotations

Section: 21.4.6 [\[meta.reflection.names\]](#) **Status:** [Immediate](#) **Submitter:** Jakub Jelinek **Opened:** 2025-11-18 **Last modified:** 2026-03-25

Priority: 1

View all other [issues](#) in [\[meta.reflection.names\]](#).

Discussion:

The `std::meta::has_identifier` metafunction doesn't say what it returns for an annotation. An annotation is not an entity, so it's not an unnamed entity, and it's also not a type, type alias, function, or any of the other things in the list.

We should add an item to the list for annotations, or just add a catch-all "Otherwise false" to the end of the list.

[2025-11-26; Reflector poll.]

Set priority to 1 after reflector poll.

'value' and 'object' are missing too.

[Croydon 2026-03-25; LWG adds wording]

[Croydon 2026-03-25; move to Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 21.4.6 [\[meta.reflection.names\]](#) as indicated:

`constexpr bool has_identifier(info r);`

- 1- *Returns*:
 - [...]
 - Otherwise, **if** *r* represents a data member description (T,N,A,W,NUA) (11.4.1 [\[class.mem.general\]](#)); true if *N* is not `⊥`. Otherwise, false.

— Otherwise, false.

4485. Move specification for `task::stop_token_type`

Section: 33.13.6.4 [task.state] Status: [Immediate](#) Submitter: Tomasz Kamiński Opened: 2025-12-08 Last modified: 2026-03-27

Priority: Not Prioritized

Discussion:

Addresses [US 249-379](#)

It is not clear what bullet 33.13.6.4 [task.state] p4.6 is – it reads like a requirement on `stop_token_type`, but if so, this paragraph is a poor place for it.

[2025-12-05 Tomasz comments]

The paragraph expresses requirements on user supplied `Environment::stop_source_type`, that needs to model *stoppable-source*, which includes *stoppable-callback-for* on associated token. We should also require that `Environment::scheduler_type` shall satisfy *scheduler*.

We also need to clarify that `stop_possible` and `stop_requested` on *prom*.token returns same value as *st* during lifetime of asynchronous operation.

[2026-01-16; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

[Croydon 2026-03-27; reopen due to conflict with LWG [4347^{\(i\)}](#).]

Previous resolution [SUPERSEDED]:

This wording is relative to [N5014](#).

1. Modify 33.13.6 [exec.task] as indicated:

-5- `allocator_type` shall meet the *Cpp17Allocator* requirements, `scheduler_type` shall model *scheduler*, and `stop_source_type` shall model *stoppable-source*.

2. Modify 33.13.6.4 [task.state] as indicated:

`void start() & noexcept;`

-4- *Effects*: *Effects*: Let *prom* be the object `handle.promise()`. Associates `STATE(prom)`, `RCVR(prom)`, and `SCHED(prom)` with `*this` as follows:

- -4.1- [...]
- -4.2- [...]
- -4.3- [...]

Let *st* be `get_stop_token(get_env(rcvr))`. Initializes *prom*.token and *prom*.source such that *during the lifetime of the asynchronous operation (33.3 [exec.async.ops]) associated with *this*

- -4.4- `prom.token.stop_requested()` returns `st.stop_requested()`;
- -4.5- `prom.token.stop_possible()` returns `st.stop_possible()`;
- -4.6- for types `Fn` and `Init` such that both `invocable<Fn>` and `constructible_from<Fn, Init>` are modeled, `stop_token_type::callback_type<Fn>` models *stoppable-callback-for<Fn, stop_token_type, Init>*.

[Croydon 2026-03-27; Jonathan provides new wording]

[Croydon 2026-03-27; Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 33.13.6.2 [task.class] as indicated:

-5- `allocator_type` shall meet the *Cpp17Allocator* requirements, `scheduler_type` shall model *scheduler*, and `stop_source_type` shall model *stoppable-source*.

4497. `std::nullopt_t` should be comparable

Section: 22.5.5 [optional.nullopt] Status: [Immediate](#) Submitter: Barry Revzin Opened: 2025-12-21 Last modified: 2026-03-27

Priority: 2

View other [active issues](#) in [optional.nullopt].

View all other [issues](#) in [optional.nullopt].

Discussion:

`std::nullopt_t` currently has no comparison operators. This prevents perfectly reasonable code from working, like `ranges::find(v, nullopt)` where `v` is a `vector<optional<T>>`, for no good reason.

Additionally, `optional<T>` has the full set of comparisons. But `optional<T>` is conceptually a `variant<nullopt_t, T>`, which wouldn't be comparable... because of `nullopt_t`. Other empty types like `tuple<>` and `monostate` are also comparable.

Proposed resolution: Add a defaulted member operator `<=>` to `nullopt_t`.

[2026-02-18; Reflector poll.]

Set priority to 2 after reflector poll.

Discussion whether this needs LEWG approval. Paper [P2405](#), for which the `nullopt` part received LEWG support, is relevant here.

[2026-03-26; Tim provides wording]

[Croydon 2026-03-27; Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Edit 22.5.5 [\[optional.nullopt\]](#), as indicated:

```
struct nullopt_t{see below};
inline constexpr nullopt_t nullopt(unspecified);
```

-1- [...]

-2- Type `nullopt_t` **shall** does not have a default constructor or an initializer-list constructor, and **shall not be** is not an aggregate. `nullopt_t` models **copyable and three way comparable<strong_ordering>**.

4504. Wording problem in {simple_}counting_scope

Section: 33.14.2.2.3 [\[exec.simple.counting.mem\]](#) **Status:** [Immediate](#) **Submitter:** Ian Petersen **Opened:** 2025-12-24 **Last modified:** 2026-03-26

Priority: 2

Discussion:

I discovered while implementing [P3149 + P3815 in NVIDIA's stdexec](#) that there's a bug in 33.14.2.2.3 [\[exec.simple.counting.mem\]](#), paragraph 9. The status quo says:

```
template<class State>
bool start-join-sender(State& st) noexcept;
```

-9- *Effects:* If *state* is

- (9.1) — *unused*, *unused-and-closed*, or *joined*, then changes *state* to *joined* and returns true;
- (9.2) — *open* or *open-and-joining*, then changes *state* to *open-and-joining*, registers *st* with **this* and returns false;
- (9.3) — *closed* or *closed-and-joining*, then changes *state* to *closed-and-joining*, registers *st* with **this* and returns false.

The problem is that if either bullet 9.2 or 9.3 applies when *count* happens to be zero then there's nothing to guarantee that `st.complete()` is ever invoked, leading to deadlock unless another sender is associated and disassociated with the scope (because the subsequent disassociation will trigger the join-sender's completion).

One fix is to change bullet 9.1 of the *Effects* element such that *start-join-sender* changes *state* to *joined* and returns true if *count* is zero, regardless of the value of *state*; bullets 9.2 and 9.3 can be left alone. I might add a note that points out that, when *state* is *unused*, *unused-and-closed*, or *joined*, *count* must be zero.

Another fix would be to delete bullet 9.1, to modify bullets 9.2 and 9.3 by adding "and *count* is greater than zero", and to add a trailing bullet that says "otherwise, changes *state* to *joined* and returns true."

[2026-02-18; Reflector poll.]

Set priority to 2 after reflector poll.

[2026-03-25; Tomasz provides updated wording.]

[Croydon 2026-03-26; move to Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 33.14.2.2.3 [\[exec.simple.counting.mem\]](#) as indicated:

```
template<class State>
bool start-join-sender(State& st) noexcept;
```

-9- Effects: If `count` is zero, then changes `state` to `joined` and returns `true`. Otherwise, if `state` is:

(9.1) — ~~unused, unused-and-closed, or joined, then changes `state` to `joined` and returns `true`;~~

(9.2) — `open` or `open-and-joining`, then changes `state` to `open-and-joining`, registers `st` with `*this` and returns `false`;

(9.3) — `closed` or `closed-and-joining`, then changes `state` to `closed-and-joining`, registers `st` with `*this` and returns `false`.

4506. `source_location` is explicitly unspecified if is constexpr or not

Section: 17.8.2 [\[support.srcloc.class\]](#) Status: [Immediate](#) Submitter: Hana Dusíková Opened: 2025-12-27 Last modified: 2026-03-27

Priority: 2

Discussion:

Basically in 17.8.2.1 [\[support.srcloc.class.general\]](#) following p1 there is a note from Jens from six years ago which says (emphasis mine):

[Note 1: The intent of `source_location` is to have a small size and efficient copying. It is unspecified whether the copy/move constructors and the copy/move assignment operators are trivial **and/or constexpr**. — end note]

But also reflection's `std::meta::source_location_of` returns it by value and is `constexpr`. This means `source_location` needs to be specified to `constexpr`. And good news is ... all three major implementations have it `constexpr` implicitly.

Options are (from my perspective) to remove that part of the note "and/or constexpr" and just call it a day, or add

```
constexpr source_location(const source_location&) = default;
constexpr source_location(source_location&&) noexcept = default;
constexpr source_location& operator=(const source_location&) = default;
constexpr source_location& operator=(source_location&&) noexcept = default;
```

AFAIK this is how to explicitly say it must be `constexpr` but it can still be trivial, see [demo](#).

[2026-02-18; Reflector poll.]

Set priority to 2 after reflector poll.

[2026-03-26; Tim adds wording]

[Croydon 2026-03-27; move to Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

[Drafting note: 16.3.3.5 [\[functions.within.classes\]](#) can be read to say that the functions are `constexpr` already, but given the note that contradicts this interpretation, explicit declarations can't hurt. — end drafting note]

1. Modify 17.8.2.1 [\[support.srcloc.class.general\]](#) as indicated:

```
namespace std {
    struct source_location {
        // source location construction
        static constexpr source_location current() noexcept;
        constexpr source_location() noexcept;

        constexpr source_location(const source_location&) noexcept = default;
        constexpr source_location& operator=(const source_location&) noexcept = default;

        // source location field access
        constexpr uint_least32_t line() const noexcept;
        constexpr uint_least32_t column() const noexcept;
        constexpr const char* file_name() const noexcept;
        constexpr const char* function_name() const noexcept;

    private:
        uint_least32_t line_;           // exposition only
        uint_least32_t column_;         // exposition only
        const char* file_name_;         // exposition only
        const char* function_name_;     // exposition only
    };
}
```

- 1- The type `source_location` meets the `Cpp17DefaultConstructible`, `Cpp17CopyConstructible`, `Cpp17CopyAssignable`, `Cpp17Swappable`, and `Cpp17Destructible` requirements (16.4.4.2 [\[utility.arg.requirements\]](#), 16.4.4.3 [\[swappable.requirements\]](#), [models](#) `semiregular`, `is_nothrow_swappable_v<source_location>` is true. All of the following conditions are true:

(1.1) — `is_nothrow_move_constructible_v<source_location>`

(1.2) — ~~is_nothrow_move_assignable_v<source_location>~~

(1.3) — ~~is_nothrow_swappable_v<source_location>~~

[Note 1: The intent of `source_location` is to have a small size and efficient copying. It is unspecified whether the copy/move constructors and the copy/move assignment operators are trivial ~~and/or constexpr~~. — end note]

4532. Imprecise `std::polymorphic` wording seems to imply slicing

Section: 20.4.2.3 [\[polymorphic.ctor\]](#), 20.4.2.5 [\[polymorphic.assign\]](#), 20.4.2.4 [\[polymorphic.dtor\]](#) **Status:** [Immediate](#) **Submitter:** Fraser Gordon **Opened:** 2026-02-27 **Last modified:** 2026-03-25

Priority: 2

Discussion:

The wording for some member functions of `std::polymorphic` is imprecise, leading to plausible interpretations that undermine the purpose of the type (i.e causing slicing on copy/move).

The wording on most copy/assign methods includes a clause "of type `U`, where `U` is the type of the owned object in `other`" but the copy/move assignment methods are missing this, which allows a reading where `allocator_traits<Allocator>::construct` is called to construct an instance of `value_type` instead of `U`.

A similar reading of the destructor effects would cause `~value_type()` to be used instead of `~U()`, so that should also be cleaned up.

Finally, the last sentence of the effects of one of the constructors (20.4.2.3 [\[polymorphic.ctor\]](#) p11) doesn't make sense in context ("constructs an object of type `polymorphic`, considering the owned object in `other` as an rvalue"). As written, this is creating a new `polymorphic` object and doing nothing with it when the intended effect would be to construct a new owned object.

[2026-03-25; Reflector poll.]

Set priority to 2 after reflector poll.

[Croydon 2026-03-25; move to Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 20.4.2.3 [\[polymorphic.ctor\]](#) as indicated:

```
constexpr polymorphic(allocator_arg_t, const Allocator& a, polymorphic&& other)
noexcept(allocator_traits<Allocator>::is_always_equal::value);
```

-11- *Effects:* `alloc` is direct-non-list-initialized with `a`. If `other` is valueless, `*this` is valueless. Otherwise, if `alloc == other.alloc` is true, either constructs an object of type `polymorphic` that owns the owned object of `other`, making `other` valueless; or, owns an object of the same type constructed from the owned object of `other` considering that owned object as an rvalue. Otherwise, if `alloc != other.alloc` is true, constructs an object of type `polymorphic`, considering owned object of type `U`, where `U` is the type of the owned object in `other`, with the owned object in `other` as an rvalue, using the allocator `alloc`.

2. Modify 20.4.2.5 [\[polymorphic.assign\]](#) as indicated:

```
constexpr polymorphic& operator=(const polymorphic& other);
```

-1- *Mandates:* `T` is a complete type.

-2- *Effects:* If `addressof(other) == this` is true, there are no effects. Otherwise:

(2.1) — [...]

(2.2) — If `other` is not valueless, a new owned object of type `U`, where `U` is the type of the owned object in `other`, is constructed in `*this` using `allocator_traits<Allocator>::construct` with the owned object from `other` as the argument, using either the allocator in `*this` or the allocator in `other` if the allocator needs updating.

(2.3) — [...]

(2.4) — [...]

```
constexpr polymorphic& operator=(polymorphic&& other)
noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
allocator_traits<Allocator>::is_always_equal::value);
```

-5- *Mandates:* If `allocator_traits<Allocator>::propagate_on_container_move_assignment::value` is false and `allocator_traits<Allocator>::is_always_equal::value` is false, `T` is a complete type.

-6- *Effects:* If `addressof(other) == this` is true, there are no effects. Otherwise:

(6.1) — [...]

(6.2) — [...]

(6.3) — [...]

(6.4) — Otherwise, constructs a new owned object of type U, where U is the type of the owned object in other, with the owned object of other as the argument as an rvalue, using the allocator in *this.

(6.5) — [...]

(6.6) — [...]

3. Modify 20.4.2.4 [\[polymorphic.dtor\]](#) as indicated:

```
constexpr ~polymorphic();
```

-1- *Mandates:* T is a complete type.

-2- *Effects:* If *this is not valueless, ~~destroys the owned object using allocator_traits<Allocator>::destroy and calls allocator_traits<Allocator>::destroy(p), where p is a pointer of type U* to the owned object and U is the type of the owned object;~~ then the storage is deallocated.

4533. not_fn<f> is unimplementable

Section: 22.10.13 [\[func.not.fn\]](#), 22.10.14 [\[func.bind.partial\]](#) **Status:** [Immediate](#) **Submitter:** Tim Song **Opened:** 2026-03-05 **Last modified:** 2026-03-25

Priority: Not Prioritized

View other [active issues](#) in [\[func.not.fn\]](#).

View all other [issues](#) in [\[func.not.fn\]](#).

Discussion:

The current specification of not_fn<f> says that the perfect forwarding call wrapper it returns does not have state entities. 22.10.4 [\[func.require\]](#)p8 says that such wrappers "returned by a given standard library function template have the same type if the types of their corresponding state entities are the same." That condition is trivially true.

It follows that not_fn<f>() must return the same type regardless of the value or type of f. This is, as far as I know, impossible to implement, and certainly not the intent.

bind_front<f> and bind_back<f> suffer from the same problem.

[Croydon 2026-03-24; move to Immediate.]

[2026-03-24 Tim replaces constant_arg with cw in light of [P3948](#).]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 22.10.3 [\[func.def\]](#) as indicated:

-5- A *call wrapper* type is a type that holds a target object, which is either a callable object or an object representing a callable object, and supports a call operation that forwards to that callable object.

-6- [...]

~~-7- A target object is the callable object held by a call wrapper.~~

-8- A call wrapper type may additionally hold a sequence of objects and references that may be passed as arguments to the target callable object. These entities are collectively referred to as *bound argument entities*.

2. Modify 22.10.13 [\[func.not.fn\]](#) as indicated:

```
template<auto f> constexpr unspecified not_fn() noexcept;
```

-6- [...]

-7- [...]

-8- *Returns:* A perfect forwarding call wrapper (22.10.4 [\[func.require\]](#)) ~~g that does not have state entities~~ whose target object is a copy of cw<f>, and ~~has the whose~~ call pattern is !invoke(f, call_args...).

3. Modify 22.10.14 [\[func.bind.partial\]](#) as indicated:

```
template<auto f, class... Args>
constexpr unspecified bind_front(Args&&... args);
template<auto f, class... Args>
constexpr unspecified bind_back(Args&&... args);
```

-6- [...]

-7- [...]

-8- [...]

-9- Returns: A perfect forwarding call wrapper (22.10.4 [\[func.require\]](#)) g that does not have a target object whose target object is a copy of cw<f>, and has the whose call pattern is:

(2.1) — invoke(f, bound_args..., call_args...) for a bind_front invocation, or

(2.2) — invoke(f, call_args..., bound_args...) for a bind_back invocation.

4537. Improve define_static_array

Section: 21.4.3 [\[meta.define.static\]](#) Status: [Immediate](#) Submitter: Hewill Kang Opened: 2026-03-09 Last modified: 2026-03-27

Priority: 2

View other [active issues](#) in [\[meta.define.static\]](#).

View all other [issues](#) in [\[meta.define.static\]](#).

Discussion:

define_static_array can transform any input range, even a non-contiguous one, into a span.

However, the returned span always has a dynamic extent. If the size of the input range is known at compile time, returning a static span seems reasonable and may offer slight runtime efficiency.

[2026-03-25; Reflector poll.]

Set priority to 2 after reflector poll.

[Croydon 2026-03-25; move to Immediate.]

Previous resolution [SUPERSEDED]:

This wording is relative to [N5032](#).

1. Modify 21.4.1 [\[meta.syn\]](#), header <meta> synopsis, as indicated:

```
#include <compare>           // see 17.12.1 \[compare.syn\]
#include <initializer_list>   // see 17.11.2 \[initializer.list.syn\]

namespace std {
    [...]
    // 21.4.3 \[meta.define.static\], promoting to static storage
    [...]
    template<ranges::input_range R>
        constexpr span<const ranges::range_value_t<R>, see below> define_static_array(R&& r);
    [...]
}
```

2. Modify 21.4.3 [\[meta.define.static\]](#) as indicated:

```
template<ranges::input_range R>
    constexpr span<const ranges::range_value_t<R>, see below> define_static_array(R&& r);
```

-16- Effects: Equivalent to:

```
using T = ranges::range_value_t<R>;
meta::info array = meta::reflect_constant_array(r);
if (meta::is_array_type(meta::type_of(array))) {
    return span<const T, see below>(meta::extract<const T*>(array), meta::extent(meta::type_of(array)));
} else {
    return span<const T, see below>();
}
```

[?- Remarks: The second template argument of the returned span type is static cast<size_t>\(ranges::size\(r\)\) if ranges::size\(r\) is a constant expression, and dynamic extent otherwise.](#)

[2026-03-26; Tim reopens and provides revised wording]

The else branch is ill-formed when the returned has non-zero static extent.

[Croydon 2026-03-27; move to Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 21.4.1 [\[meta.syn\]](#), header <meta> synopsis, as indicated:

```
#include <compare>           // see 17.12.1 \[compare.syn\]
#include <initializer_list>   // see 17.11.2 \[initializer.list.syn\]

namespace std {
```

```
[...]
// 21.4.3 [meta.define.static], promoting to static storage
[...]
template<ranges::input_range R>
constexpr span<const ranges::range_value_t<R>, see below> define_static_array(R&& r);
[...]
```

2. Modify 21.4.3 [meta.define.static] as indicated:

```
template<ranges::input_range R>
constexpr span<const ranges::range_value_t<R>, see below> define_static_array(R&& r);
```

-16- *Effects*: Equivalent to:

```
using T = ranges::range_value_t<R>;
meta::info array = meta::reflect_constant_array(r);
if (meta::is_array_type(meta::type_of(array))) {
    return span<const T, see below>(meta::extract<const T*>(array), meta::extent(meta::type_of(array)));
} else {
    return span<const T, see below>(static cast<const T*>(nullptr), 0);
}
```

Remarks: The second template argument of the returned span type is `static cast<size_t>(ranges::size(r))` if `ranges::size(r)` is a constant expression, and dynamic extent otherwise.

4540. *future-senders* returned from `spawn_future` do not forward stop requests to spawned work

Section: 33.9.12.18 [exec.spawn.future] **Status:** Immediate **Submitter:** Ian Petersen **Opened:** 2026-03-10 **Last modified:** 2026-03-27

Priority: 1

Discussion:

The wording that describes `spawn_future` (specifically 33.9.12.18 [exec.spawn.future] paragraph 13 and paragraph 14) does not capture a critical element of the design intent originally expressed in P3149R11, section 5.5.

P3149R11, section 5.5 reads in part,

When `fsop` is started, if `fsop` receives a stop request from its receiver before the eagerly-started work has completed then an attempt is made to abandon the eagerly-started work. Note that it's possible for the eagerly-started work to complete while `fsop` is requesting stop; once the stop request has been delivered, either `fsop` completes with the result of the eagerly-started work if it's ready, or it completes with `set_stopped()` without waiting for the eagerly-started work to complete.

In the foregoing, `fsop` is the name of an operation state constructed by connecting a *future-sender* (i.e. a sender returned from `spawn_future`) to a receiver.

Paragraphs 13 and 14 of 33.9.12.18 [exec.spawn.future] describe the behaviour of the *future-sender* returned from `spawn_future` in terms of the *basic-sender* machinery like so:

-13- The exposition-only class template *impls-for* (33.9.2 [exec.snd.expos]) is specialized for `spawn_future_t` as follows:

```
namespace std::execution {
    template<>
    struct impls-for<spawn_future_t> : default-impls {
        static constexpr auto start = see below; // exposition only
    };
}
```

-14- The member `impls-for<spawn_future_t>::start` is initialized with a callable object equivalent to the following lambda:

```
[](auto& state, auto& rcvr) noexcept -> void {
    state->consume(rcvr);
}
```

Since there's no specification for the behaviour of `std::execution::impls-for<spawn_future_t>::get-state`, the behaviour is the default provided by `std::execution::default-impls::get-state`, which just returns the "data" member of the original result of *make-sender*. In this case, that is the object named `u` defined in 33.9.12.18 [exec.spawn.future] bullet 16.2, which is an instance of a specialization of `std::unique_ptr`. There is therefore no wording to require that a *future-sender* that has been connected and started take any action in response to stop requests received through the receiver to which it was connected, contrary to the LEWG-approved design intent.

An implementation that addresses this issue is included in [stdexec PR 1713](#), specifically in commit [5209ffdc9a3badf0079746b5578c12a1d0da4f](#), which is just the difference between the current wording and the intended design.

[2026-03-25; Reflector poll.]

Set priority to 1 after reflector poll.

Previous resolution [SUPERSEDED]:

This wording is relative to [N5032](#).

1. Modify 33.9.12.18 [exec.spawn.future] as indicated:

-2- The name `spawn_future` denotes a customization point object. For subexpressions `sndr`, `token`, and `env`,

[...]

If any of `sender<Sndr>`, `scope_token<Token>`, or `queryable<Env>` are not satisfied, the expression `spawn_future(sndr, token, env)` is ill-formed.

-?- Let `try-cancelable` be the exposition-only class:

```
namespace std::execution {
    struct try_cancelable { // exposition only
        virtual void try_cancel() noexcept = 0; // exposition only
    };
}
```

-3- Let `spawn-future-state-base` be the exposition-only class template: [...]

```
namespace std::execution {
    template<class Completions>
    struct spawn-future-state-base; // exposition only

    template<class... Sigs>
    struct spawn-future-state-base<completion_signatures<Sigs...>> { // exposition only
        : try_cancelable {
            using variant-t = see below; // exposition only
            variant-t result; // exposition only
            virtual void complete() noexcept = 0; // exposition only
        };
    };
}
```

[...]

-7- Let `spawn-future-state` be the exposition-only class template:

```
namespace std::execution {
    template<class Alloc, scope_token Token, sender Sender, class Env>
    struct spawn-future-state // exposition only
        : spawn-future-state-base<completion_signatures_of_t<future-spawned-sender<Sender, Env>>> {
        [...]
        void complete() noexcept override; // exposition only
        void consume(receiver auto& rcvr) noexcept; // exposition only
        void abandon() noexcept; // exposition only
        void try_cancel() noexcept override { // exposition only
            ssource.request_stop();
            try-set-stopped();
        }
        void try-set-stopped() noexcept; // exposition only
        [...]
    };
    [...]
}
```

-8- For purposes of determining the existence of a data race, `complete`, `consume`, `try-set-stopped`, and `abandon` behave as atomic operations (6.10.2 [intro.multithread]). These operations on a single object of a type that is a specialization of `spawn-future-state` appear to occur in a single total order.

`void complete() noexcept;`

-9- *Effects:*

(9.1) — No effects if this invocation of `complete` happens before an invocation of `consume`, `try-set-stopped`, or `abandon` on `*this`;

(9.2) — otherwise, if an invocation of `consume` on `*this` happens before this invocation of `complete` and no invocation of `try-set-stopped` on `*this` happened before this invocation of `complete` then there is a receiver, `rcvr`, registered and that receiver is deregistered and completed as if by `consume(rcvr)`;

(9.3) — otherwise, `destroy` is invoked.

`void consume(receiver auto& rcvr) noexcept;`

-10- *Effects:*

(10.1) — If this invocation of `consume` happens before an invocation of `complete` on `*this` and no invocation of `try-cancel` on `*this` happened before this invocation of `consume` then `rcvr` is registered to be completed when `complete` is subsequently invoked on `*this`;

(10.2) — otherwise, if this invocation of `consume` happens after an invocation of `try-cancel` on `*this` and no invocation of `complete` on `*this` happened before this invocation of `consume` then `rcvr` is completed as if by `set_stopped(std::move(rcvr))`;

(10.2) — otherwise, `rcvr` is completed as if by: [...]

`void try_cancel() noexcept;`

-?- *Effects:*

(?1) — No effects if this invocation of *try-cancel* happens after an invocation of *complete* on **this*;

(?2) — otherwise, if this invocation of *try-cancel* happens before an invocation of *consume* on **this* then invokes *ssource.request_stop()*;

(?3) — otherwise,

(?3.1) — invokes *ssource.request_stop()*, and

(?3.2) — if there is a receiver, *rcvr*, still registered then that receiver is deregistered and completed as if by *set_stopped(std::move(rcvr))*.

[*Note: an invocation of *complete* on **this* may have happened after the just-described invocation of *ssource.request_stop()* and happened before the check to see if there is a receiver still registered; if so, it would have deregistered and completed the previously-registered receiver. Only one of *try-cancel* or *complete* completes the registered receiver and no data races are introduced between the two invocations. — end note*]

```
void abandon() noexcept;
```

```
[...]
```

```
void destroy() noexcept;
```

-12- *Effects*: Equivalent to:

```
auto associated = std::move(this->associated);
{
    using traits = allocator_traits<Alloc>::template rebind_traits<spawn-future-state>;
    typename traits::allocator_type alloc(std::move(this->alloc));
    traits::destroy(alloc, this);
    traits::deallocate(alloc, this, 1);
}
```

-?- Let *future-operation* be the exposition-only class template:

```
namespace std::execution {
    template<class StatePtr, class Rcvr>
    struct future-operation { // exposition only
        struct callback { // exposition only
            try-cancelable* state; // exposition only
        };
        void operator()() noexcept {
            state->try-cancel();
        };
        using stop-token-t = // exposition only
            stop token of t<env of t<Rcvr>>;
        using stop-callback-t = // exposition only
            stop callback for t<stop-token-t, callback>;
        struct receiver { // exposition only
            using receiver concept = receiver t;
            future-operation* op; // exposition only
        };
        template<class... T>
        void set_value(T&&... ts) && noexcept {
            op->set-complete<set value t>(std::forward<T>(ts)...);
        }
        template<class E>
        void set_error(E&& e) && noexcept {
            op->set-complete<set error t>(std::forward<E>(e));
        }
        void set_stopped() && noexcept {
            op->set-complete<set stopped t>();
        }
        env of t<Rcvr> get_env() const noexcept {
            return op->rcvr.get_env();
        }
    };
    Rcvr rcvr; // exposition only
    union {
        StatePtr state; // exposition only
        receiver inner; // exposition only
    };
    union {
        stop-callback-t stopCallback; // exposition only
    };
    future-operation(StatePtr state, Rcvr rcvr) noexcept // exposition only
        : rcvr(std::move(rcvr))
}
```

```

    {
        construct at(addressof(state), std::move(state));
    }

    future-operation(future-operation&&) = delete;

    ~future-operation() {
        destroy_at(addressof(state));
    }

    void run() & noexcept { // exposition only
        constexpr bool nothrow =
            is_nothrow_constructible_v<stop-callback-t, stop-token-t, callback>;
        try {
            construct at(addressof(stopCallback), _get_stop_token(rcvr), callback(state.get()));
        }
        catch (...) {
            if constexpr (!nothrow) {
                set_error(std::move(rcvr), current_exception());
            }
            return;
        }
    }

    auto* state = state.release();
    destroy_at(addressof(state));
    construct at(addressof(inner), this);
    state->consume(inner);
}

template<class CP0, class... T>
void set-complete(T&&... ts) noexcept { // exposition only
    destroy_at(addressof(stopCallback));
    destroy_at(addressof(inner));
    construct at(addressof(state), nullptr);
    CP0{}(std::move(rcvr), std::forward<T>(ts)...);
}
};
}

```

-13- The exposition-only class template *impls-for* (33.9.2 [\[exec.snd.expos\]](#)) is specialized for `spawn_future_t` as follows:

```

namespace std::execution {
    template<>
    struct impls-for<spawn_future_t> : default-impls {
        static constexpr auto start = see below; // exposition only
        static constexpr auto get-state = see below; // exposition only
    };
}

```

-14- The member *impls-for*<spawn_future_t>::start is initialized with a callable object equivalent to the following lambda:

```

[](auto& state, auto& rcvr) noexcept -> void {
    state.run() -> consume(rcvr);
}

```

-?- The member *impls-for*<spawn_future_t>::get-state is initialized with a callable object equivalent to the following lambda:

```

[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept {
    auto& [_, data] = sndr;
    using state_ptr = remove_cvref_t<decltype(data)>;
    return future-operation<state_ptr, Rcvr>(std::move(data), std::move(rcvr));
}

```

[2026-03-25; Tomasz provides updated wording.]

The updated wording removes the union inside *future-operation*.

[Croydon 2026-03-27; Status changed: New → Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 33.9.12.18 [\[exec.spawn.future\]](#) as indicated:

-2- The name `spawn_future` denotes a customization point object. For subexpressions `sndr`, `token`, and `env`,

[...]

If any of `sender`<`Sndr`>, `scope_token`<`Token`>, or `queryable`<`Env`> are not satisfied, the expression `spawn_future(sndr, token, env)` is ill-formed.

-?- Let *try-cancelable* be the exposition-only class:

```

namespace std::execution {
    struct try-cancelable {                // exposition only
        virtual void try-cancel() noexcept = 0; // exposition only
    };
}

```

-3- Let *spawn-future-state-base* be the exposition-only class template: [...]

```

namespace std::execution {
    template<class Completions>
    struct spawn-future-state-base;          // exposition only

    template<class... Sigs>
    struct spawn-future-state-base<completion_signatures<Sigs...>> { // exposition only
        : try-cancelable {
            using variant-t = see below;          // exposition only
            variant-t result;                     // exposition only
            virtual void complete() noexcept = 0; // exposition only
        };
    };
}

```

[...]

-7- Let *spawn-future-state* be the exposition-only class template:

```

namespace std::execution {
    template<class Alloc, scope_token Token, sender Sender, class Env>
    struct spawn-future-state                // exposition only
    : spawn-future-state-base<completion_signatures_of_t<future-spawned-sender<Sender, Env>>> {
        [...]
        void complete() noexcept override;    // exposition only
        void consume(receiver auto& rcvr) noexcept; // exposition only
        void abandon() noexcept;              // exposition only
        void try-cancel() noexcept override { // exposition only
            ssource.request_stop();
            try-set-stopped();
        }
        void try-set-stopped() noexcept; // exposition only
        [...]
    };
    [...]
}

```

-8- For purposes of determining the existence of a data race, *complete*, *consume*, *try-set-stopped*, and *abandon* behave as atomic operations (6.10.2 [intro.multithread]). These operations on a single object of a type that is a specialization of *spawn-future-state* appear to occur in a single total order.

void *complete*() noexcept;

-9- Effects:

(9.1) — No effects if this invocation of *complete* happens before an invocation of *consume*, *try-set-stopped*, or *abandon* on **this*;

(9.2) — otherwise, if an invocation of *consume* on **this* happens before this invocation of *complete* and no invocation of *try-set-stopped* on **this* happened before this invocation of *complete* then there is a receiver, *rcvr*, registered and that receiver is *deregistered and* completed as if by *consume(rcvr)*;

(9.3) — otherwise, *destroy* is invoked.

void *consume*(receiver auto& rcvr) noexcept;

-10- Effects:

(10.1) — If this invocation of *consume* happens before an invocation of *complete* on **this* and no invocation of *try-set-stopped* on **this* happened before this invocation of *consume* then *rcvr* is registered to be completed when *complete* is subsequently invoked on **this*;

(10.?) — otherwise, if this invocation of *consume* happens after an invocation of *try-set-stopped* on **this* and no invocation of *complete* on **this* happened before this invocation of *consume* then *rcvr* is completed as if by *set stopped(std::move(rcvr))*;

(10.2) — otherwise, *rcvr* is completed as if by:

```

std::move(this->result).visit(
    [&rcvr](auto&& tuple) noexcept {
        if constexpr (!same_as<remove_reference_t<decltype(tuple)>, monostate>) {
            apply([&rcvr](auto cpo, auto&&... vals) {
                cpo(std::move(rcvr), std::move(vals)...);
            }, std::move(tuple));
        }
    });
destroy();

```

void *try-set-stopped*() noexcept;

-?- Effects:

(?.1) — If an invocation of *consume* on **this* happens before this invocation of *try-set-stopped* and no invocation of *complete* on **this* happened before this invocation of *try-set-stopped* then there is a receiver, *rcvr*, registered and that receiver is deregistered and completed as if by *set_stopped(std::move(rcvr)), destroy()*:

(?.2) — otherwise, no effects.

void *abandon*() noexcept;

[...]

void *destroy*() noexcept;

-12- *Effects*: Equivalent to:

```
auto associated = std::move(this->associated);
{
    using traits = allocator_traits<Alloc>::template rebind_traits<spawn-future-state>;
    typename traits::allocator_type alloc(std::move(this->alloc));
    traits::destroy(alloc, this);
    traits::deallocate(alloc, this, 1);
}
```

-?.- Let *future-operation* be the exposition-only class template:

```
namespace std::execution {
    template<class StatePtr, class Rcvr>
    struct future-operation { // exposition only
        struct callback { // exposition only
            try-cancelable* state; // exposition only
        };
        void operator()() noexcept {
            state->try-cancel();
        };
    };

    using stop-token-t = // exposition only
        stop token of t<env of t<Rcvr>>;

    using stop-callback-t = // exposition only
        stop callback for t<stop-token-t, callback>;

    struct receiver { // exposition only
        using receiver concept = receiver t;
        future-operation* op; // exposition only
    };

    template<class... T>
    void set_value(T&&... ts) && noexcept {
        op->set-complete<set value t>(std::forward<T>(ts)...);
    }

    template<class E>
    void set_error(E&& e) && noexcept {
        op->set-complete<set error t>(std::forward<E>(e));
    }

    void set_stopped() && noexcept {
        op->set-complete<set stopped t>();
    }

    env of t<Rcvr> get_env() const noexcept {
        return execution::get_env(op->rcvr);
    }

    Rcvr rcvr; // exposition only
    StatePtr state; // exposition only
    receiver inner; // exposition only
    optional<stop-callback-t> stopCallback; // exposition only

    future-operation(StatePtr state, Rcvr rcvr) noexcept // exposition only
        : rcvr(std::move(rcvr)), state(std::move(state)), inner(this)
    {}

    future-operation(future-operation&&) = delete;

    void run() && noexcept { // exposition only
        constexpr bool nothrow =
            is_nothrow_constructible_v<stop-callback-t, stop-token-t, callback>;
        try {
            stopCallback.emplace(get_stop_token(rcvr), callback(state.get()));
        }
        catch (...) {
            if constexpr (!nothrow) {
                set_error(std::move(rcvr), current_exception());
            }
            return;
        }

        state.release()->consume(inner);
    }
}
```

```

template<class CP0, class... T>
void set_complete(T&&... ts) noexcept { // exposition only
    stopCallback.reset();
    CP0{}(std::move(rcvr), std::forward<T>(ts)...);
}
};
}

```

-13- The exposition-only class template *impls-for* (33.9.2 [\[exec.snd.expos\]](#)) is specialized for *spawn_future_t* as follows:

```

namespace std::execution {
    template<>
    struct impls_for<spawn_future_t> : default_impls {
        static constexpr auto start = see below; // exposition only
        static constexpr auto get_state = see below; // exposition only
    };
}

```

-14- The member *impls-for*<spawn_future_t>::*start* is initialized with a callable object equivalent to the following lambda:

```

[](auto& state, auto& rcvr) noexcept -> void {
    state.run()->consume(rcvr);
}

```

-?- The member *impls-for*<spawn_future_t>::*get_state* is initialized with a callable object equivalent to the following lambda:

```

[]<class Sndr, class Rcvr>(Sndr sndr, Rcvr& rcvr) noexcept {
    auto& [, data] = sndr;
    using state_ptr = remove_cvref_t<decltype(data)>;
    return future_operation<state_ptr, Rcvr>(std::move(data), std::move(rcvr));
}

```

4544. Parallel overload of *ranges::set_difference* should return *in_in_out_result*

Section: 26.8.7.5 [\[set.difference\]](#) Status: [Immediate](#) Submitter: Ruslan Arutyunyan Opened: 2026-03-18 Last modified: 2026-03-25

Priority: Not Prioritized

Discussion:

Serial *ranges::set_difference* returns *in_out_result*, even though it takes two input ranges. This is probably fine because it assumes that the output size is always sufficient for the algorithms to write all necessary elements. We only need to go to the end of the first input and we usually do not care where the second input stops even if we did not fully traverse it. However, for parallel range algorithms the output size can be insufficient to have every single element from input that fits. When the output has smaller size than the number of elements it is supposed to take (based on the input), it should be useful for users to understand where stop points are for both first and second inputs. Thus, we need to return *in_in_out_result*. The algorithm should return the stop point of the first input, which is the first element that has to be written to the output but doesn't fit and it should also return the stop point of the second input, which is the element that was compared with the input1 element that does not fit. Otherwise, the algorithm returns an iterator that compares equal with *last2*, for the second input, meaning that it is already exhausted.

This design for parallel range *set_difference* will also be beneficial if and when we are going to add serial range algorithms with the range-as-the-output-design aspect. For the behavior illustration purposes please look at the serial implementation of such algorithm:

```

template <class In1, class In2, class Out>
using set_difference_truncated_result = std::ranges::in_in_out_result<In1, In2, Out>;

struct set_difference_fn
{
    template<std::input_iterator I1, std::sentinel_for<I1> S1,
            std::input_iterator I2, std::sentinel_for<I2> S2,
            std::input_iterator O, std::sentinel_for<O> OutS,
            class Comp = std::ranges::less,
            class Proj1 = std::identity, class Proj2 = std::identity>
    requires std::mergeable<I1, I2, O, Comp, Proj1, Proj2>
    constexpr set_difference_truncated_result<I1, I2, O>
    operator()(I1 first1, S1 last1, I2 first2, S2 last2,
              O result, OutS result_last, Comp comp = {},
              Proj1 proj1 = {}, Proj2 proj2 = {}) const
    {
        while (!(first1 == last1 || first2 == last2))
        {
            if (std::invoke(comp, std::invoke(proj1, *first1), std::invoke(proj2, *first2)))
            {
                if (result == result_last)
                    break;

                *result = *first1;
                ++first1;
                ++result;
            }
            else if (std::invoke(comp, std::invoke(proj2, *first2),
                                std::invoke(proj1, *first1)))
            {
                ++first2;
            }
            else
            {
                ++first1;
                ++first2;
            }
        }
    }
}

```

```

    }
}

while (first1 != last1 && result != result_last)
{
    *result = *first1;
    ++first1;
    ++result;
}

return {first1, first2, result};
}

template<std::ranges::input_range R1, std::ranges::input_range R2,
        std::ranges::input_range OR, class Comp = std::ranges::less,
        class Proj1 = std::identity, class Proj2 = std::identity>
    requires std::mergeable<std::ranges::iterator_t<R1>, std::ranges::iterator_t<R2>,
std::ranges::iterator_t<OR>, Comp, Proj1, Proj2>
    constexpr set_difference_truncated_result<std::ranges::borrowed_iterator_t<R1>,
        std::ranges::borrowed_iterator_t<R2>, formatPainter
        std::ranges::borrowed_iterator_t<OR>>

operator()(R1&& r1, R2&& r2, OR&& result, Comp comp = {},
        Proj1 proj1 = {}, Proj2 proj2 = {}) const
{
    return (*this)(std::ranges::begin(r1), std::ranges::end(r1),
        std::ranges::begin(r2), std::ranges::end(r2),
        std::ranges::begin(result), std::ranges::end(result),
        std::move(comp), std::move(proj1), std::move(proj2));
}
};

inline constexpr set_difference_fn set_difference{};

```

For instance, if the first input is `std::vector v1{1, 2, 3, 4, 5, 6, 7}`, the second input is `std::vector v2{3, 4, 7}`, and the output is `std::vector<int> output(3)` then for the call

```
auto [in1, in2, out] = std::ranges::set_difference(v1, v2, output);
```

- output is {1, 2, 5}
- in1 points to 6 in v1
- in2 points to 7 in v2
- out points to output.end()

Previous resolution [SUPERSEDED]:

This wording is relative to [N5032](#).

1. Modify 26.4 [\[algorithm.syn\]](#), header `<algorithm>` synopsis, as indicated:

```

[...]
namespace ranges {
    template<class I, class O>
        using set_difference_result = in_out_result<I, O>;
    template<class I1, class, I2, class O>
    using set_difference_truncated_result = in_in_out_result<I1, I2, O>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        weakly_incrementable O, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
        requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
        constexpr set_difference_result<I1, O>
            set_difference(I1 first1, S1 last1, I2 first2, S2 last2, O result,
                Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

    template<input_range R1, input_range R2, weakly_incrementable O,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
        requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
        constexpr set_difference_result<borrowed_iterator_t<R1>, O>
            set_difference(R1&& r1, R2&& r2, O result,
                Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

    template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        random_access_iterator O, sized_sentinel_for<O> OutS, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
        requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
        set_differencetruncated_result<I1, I2, O>
            set_difference(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
                O result, OutS result_last, Comp comp = {}, Proj1 proj1 = {},
                Proj2 proj2 = {}); // freestanding-deleted

    template<execution_policy Ep, sized-random-access-range R1, sized-random-access-range R2,
        sized-random-access-range OutR, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
        requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp, Proj1, Proj2>
        set_differencetruncated_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>// freestanding-deleted

```

```
}
[...]
```

2. Modify 26.8.7.5 [\[set.difference\]](#) as indicated:

```
[...]
template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        weakly_incremtable 0, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, 0, Comp, Proj1, Proj2>
constexpr ranges::set_difference_result<I1, 0>
    ranges::set_difference(I1 first1, S1 last1, I2 first2, S2 last2, 0 result,
        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2, weakly_incremtable 0,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, 0, Comp, Proj1, Proj2>
constexpr ranges::set_difference_result<borrowed_iterator_t<R1>, 0>
    ranges::set_difference(R1&& r1, R2&& r2, 0 result,
        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        random_access_iterator 0, sized_sentinel_for<0> OutS, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, 0, Comp, Proj1, Proj2>
ranges::set_difference_truncated_result<I1, I2, 0>
    ranges::set_difference(Ep&& exec, I1 first1, S1 last1,
        I2 first2, S2 last2, 0 result, OutS result_last,
        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<execution_policy Ep, sized_random_access_range R1, sized_random_access_range R2,
        sized_random_access_range OutR, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp, Proj1, Proj2>
ranges::set_difference_truncated_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, borrowed_iterator_t<OutR>>
    ranges::set_difference(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r, Comp comp = {},
        Proj1 proj1 = {}, Proj2 proj2 = {});
```

-1- Let: [...]

-2- *Preconditions*: The ranges [first1, last1) and [first2, last2) are sorted with respect to comp and proj1 or proj2, respectively. The resulting range does not overlap with either of the original ranges.

-3- *Effects*: Constructs a sorted difference between the elements from the two ranges; that is, the set of elements that are present in the range [first1, last1) but not [first2, last2). If [first1, last1) contains m elements that are equivalent to each other and [first2, last2) contains n elements that are equivalent to them, the last $\max(m - n, 0)$ elements from [first1, last1) are included in the sorted difference, in order. Of those equivalent elements, the first $\min(m, n)$ elements in both ranges are considered *skipped*. An element from the second range is also considered skipped if it compares less than the $\min(N + 1, M)^{\text{th}}$ element of the sorted difference. Copies the first N elements of the sorted difference to the range [result, result + N).

-4- *Returns*:

(4.1) — result_last for the overloads in namespace std.

(4.2) — {last1, result + N } for the overloads in namespace ranges, if N is equal to M . For non-parallel algorithm overloads in namespace ranges:

(4.2.1) — {last1, result + N }, if N is equal to M .

(4.2.2) — Otherwise, {j1, result_last}, where the iterator j1 points to the position of the element in [first1, last1) corresponding to the $(N + 1)^{\text{th}}$ element of the sorted difference.

(4.3) — Otherwise, {j1, result_last} for the overloads in namespace ranges, where the iterator j1 points to the position of the element in [first1, last1) corresponding to the $(N + 1)^{\text{th}}$ element of the sorted difference. For parallel algorithm overloads in namespace ranges:

(4.3.1) — {last1, first2 + B , result + N }, if N is equal to M , where B is the number of skipped elements in [first2, last2).

(4.3.2) — Otherwise, {first1 + A , first2 + B , result_last}, where A and B are the numbers of copied or skipped elements in [first1, last1) and [first2, last2), respectively.

[Croydon 2026-03-24; update wording after LWG review]

[Croydon 2026-03-24; move to Immediate]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 26.4 [\[algorithm.syn\]](#), header <algorithm> synopsis, as indicated:

```
[...]
namespace ranges {
    template<class I, class O>
```



```

using set_difference_result = in_out_result<I, 0>;
template<class I1, class I2, class O>
using set_difference_truncated_result = in_out_result<I1, I2, 0>;

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        weakly_incrementable O, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
constexpr set_difference_result<I1, 0>
    set_difference(I1 first1, S1 last1, I2 first2, S2 last2, O result,
        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<input_range R1, input_range R2, weakly_incrementable O,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
constexpr set_difference_result<borrowed_iterator_t<R1>, 0>
    set_difference(R1&& r1, R2&& r2, O result,
        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        random_access_iterator O, sized_sentinel_for<O> OutS, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
set_difference_truncated_result<I1, I2, 0>
    set_difference(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
        O result, OutS result_last, Comp comp = {}, Proj1 proj1 = {},
        Proj2 proj2 = {}); // freestanding-deleted

template<execution_policy Ep, sized_random_access_range R1, sized_random_access_range R2,
        sized_random_access_range OutR, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp, Proj1, Proj2>
set_difference_truncated_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>,
    borrowed_iterator_t<OutR>>
    set_difference(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r, Comp comp = {},
        Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
}
[...]
```

2. Modify 26.8.7.5 [set_difference] as indicated:

```

[...]
```

```

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        weakly_incrementable O, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
constexpr ranges::set_difference_result<I1, 0>
    ranges::set_difference(I1 first1, S1 last1, I2 first2, S2 last2, O result,
        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<input_range R1, input_range R2, weakly_incrementable O,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
constexpr ranges::set_difference_result<borrowed_iterator_t<R1>, 0>
    ranges::set_difference(R1&& r1, R2&& r2, O result,
        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        random_access_iterator O, sized_sentinel_for<O> OutS, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
ranges::set_difference_truncated_result<I1, I2, 0>
    ranges::set_difference(Ep&& exec, I1 first1, S1 last1,
        I2 first2, S2 last2, O result, OutS result_last,
        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, sized_random_access_range R1, sized_random_access_range R2,
        sized_random_access_range OutR, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp, Proj1, Proj2>
ranges::set_difference_truncated_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>,
    borrowed_iterator_t<OutR>>
    ranges::set_difference(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r, Comp comp = {},
        Proj1 proj1 = {}, Proj2 proj2 = {});
```

-1- Let: [...]

-2- *Preconditions*: The ranges [first1, last1) and [first2, last2) are sorted with respect to comp and proj1 or proj2, respectively. The resulting range does not overlap with either of the original ranges.

-3- *Effects*: Constructs a sorted difference between the elements from the two ranges; that is, the set of elements that are present in the range [first1, last1) but not [first2, last2). If [first1, last1) contains m elements that are equivalent to each other and [first2, last2) contains n elements that are equivalent to them, the last $\max(m - n, 0)$ elements from [first1, last1) are included in the sorted difference, in order. Of those equivalent elements, the first $\min(m, n)$ elements in both ranges are considered skipped. An element from the second range is also considered skipped if it compares less than the $\min(N + 1, M)$ th element of the sorted difference. Copies the first N elements of the sorted difference to the range [result, result + N).

-4- *Returns*:

(4.1) — result_last for the overloads in namespace std.

(4.2) — {last1, result + N } for the non-parallel overloads in namespace ranges, if N is equal to M .

(4.3) — Otherwise, `{j1, result_last}` for the overloads in namespace `ranges`, where the iterator `j1` points to the position of the element in `[first1, last1)` corresponding to the $(M + 1)^{\text{th}}$ element of the sorted difference. For the parallel algorithm overloads in namespace `ranges`:

(4.3.1) — `{last1, first2 + B, result + N}`, if N is equal to M , where B is the number of skipped elements in `[first2, last2)`.

(4.3.2) — Otherwise, `{ first1 + A, first2 + B, result last}`, where A and B are the numbers of copied or skipped elements in `[first1, last1)` and `[first2, last2)`, respectively.

4548. Parallel ranges::set_intersection should not do unnecessary work

Section: 26.8.7.4 [\[set.intersection\]](#) Status: [Immediate](#) Submitter: Ruslan Arutyunyan Opened: 2026-03-20 Last modified: 2026-03-25

Priority: Not Prioritized

View other [active issues](#) in `[set.intersection]`.

View all other [issues](#) in `[set.intersection]`.

Discussion:

Serial `ranges::set_intersection` returns `in_in_out_result` and it always goes to the end of both input ranges. However, when either of the input ranges reaches the end, there cannot be intersection anymore, so the algorithm could immediately stop but does not do that.

For example, if the first input is `{1}` and second input is `{1, 2, 3, 4, ..., 1000000}` it's clear that the output will be `{1}` after the first iteration of `ranges::set_intersection`. The first input is already exhausted, so the algorithm could stop and return:

- `it1` that compared equal with first `last1`,
- `it2` that points to the element equal to 2 in the second input (in other words, stop point for the second input), and
- `result_it` that points to past the last written element

Unfortunately, the algorithm does 999999 "unnecessary" operations just to calculate the iterator that compared equal to `last2`, not to compute an intersection itself.

Of course, this case is problematic only for non-random access, non-sized ranges. Still, it looks like an oversight because:

- Serial overload can work with input iterators,
- We almost certainly do not want to pessimize the algorithm execution time to just calculate `last1` or `last2`, when the algorithm could potentially stop earlier, and
- `std::set_intersection` (not the `ranges` one) stops when it hits the end of either input.

For parallel `ranges::set_intersection` we just copied the current behavior of serial `ranges::set_intersection` because parallel overload requires *sized-random-access-range*, so it's not a problem to calculate both `last1` and `last2` iterators for a constant time in this case. Then we realized that this problem exists for the serial overload.

Fixing C++20 `ranges::set_intersection` is a breaking change, unfortunately. However, we can still change the parallel overload to mimic the serial behavior that we want. This is especially beneficial if and when we add serial range algorithms with the range-as-the-output-design aspect.

The proposed resolution:

- When the output size is not sufficient to copy all necessary elements from both inputs — no changes to the specified parallel `ranges::set_intersection` behavior.
- When the output size is sufficient, change the behavior of parallel `ranges::set_intersection` to return `last1` or `last2` — whichever input range is exhausted first — and a stop point in non-exhausted input range, which is the element that is:
 - Past the last element of non-exhausted range, if it compared equal with the last written element to the output, or otherwise
 - The last element that does not compare equal to *exhausted-range-last* - 1 element.

The trade-off of the proposed behavior is that the overload with execution policy would return something different at runtime compared to the serial overload. However, if we don't apply the proposed resolution, we will lock ourselves into suboptimal `ranges::set_intersection` implementation by design, even for the future algorithms extension.

For the behavior illustration purposes please look at the serial implementation of serial `ranges::set_intersection` with range-as-the-output:

```
struct set_intersection_fn
{
    template<std::input_iterator I1, std::sentinel_for<I1> S1,
             std::input_iterator I2, std::sentinel_for<I2> S2,
             std::input_iterator O, std::sentinel_for<O> OutS,
             class Comp = std::ranges::less,
             class Proj1 = std::identity, class Proj2 = std::identity>
    requires std::mergeable<I1, I2, O, Comp, Proj1, Proj2>
    constexpr std::ranges::set_intersection_result<I1, I2, O>
    operator()(I1 first1, S1 last1, I2 first2, S2 last2,
               O result, OutS result_last, Comp comp = {},
```

```

        Proj1 proj1 = {}, Proj2 proj2 = {}) const
    {
        while (first1 != last1 && first2 != last2)
        {
            if (std::invoke(comp, std::invoke(proj1, *first1),
                           std::invoke(proj2, *first2)))
                ++first1;
            else if (std::invoke(comp, std::invoke(proj2, *first2),
                                   std::invoke(proj1, *first1)))
                ++first2;
            else
            {
                if (result == result_last)
                    break;

                *result = *first1;
                ++first1;
                ++first2;
                ++result;
            }
        }
        return {std::move(first1), std::move(first2), std::move(result)};
    }
}

template<std::ranges::input_range R1, std::ranges::input_range R2,
        std::ranges::input_range OutR, class Comp = std::ranges::less,
        class Proj1 = std::identity, class Proj2 = std::identity>
requires std::mergeable<std::ranges::iterator_t<R1>, std::ranges::iterator_t<R2>,
                        std::ranges::iterator_t<OutR>, Comp, Proj1, Proj2>
constexpr std::ranges::set_intersection_result<std::ranges::borrowed_iterator_t<R1>,
                                                std::ranges::borrowed_iterator_t<R2>,
                                                std::ranges::borrowed_iterator_t<OutR>>
operator()(R1&& r1, R2&& r2, OutR&& result, Comp comp = {},
          Proj1 proj1 = {}, Proj2 proj2 = {}) const
{
    return (*this)(std::ranges::begin(r1), std::ranges::end(r1),
                  std::ranges::begin(r2), std::ranges::end(r2),
                  std::ranges::begin(result), std::ranges::end(result),
                  std::move(comp), std::move(proj1), std::move(proj2));
}

};

inline constexpr set_intersection_fn set_intersection {};

```

For instance, if the first input is `std::vector v1{2, 4, 5}` the second input is `std::vector v2{1, 2, 3, 4, 5, 6, 7}`, and the output is `std::vector<int> output(3)` then for the call

```
auto [in1, in2, out] = std::ranges::set_difference(v1, v2, output);
```

- output is {2, 4, 5}
- in1 points to v1.end()
- in2 points to 6 in v2
- out points to output.end()

Another example: if the first input is `std::vector v1{1, 2, 3, 4}` the second input is `std::vector v2{0, 1, 2, 3, 5, 7}`, and the output is `std::vector<int> output(3)` then for the call

```
auto [in1, in2, out] = std::ranges::set_difference(v1, v2, output);
```

- output is {1, 2, 3}
- in1 points to v1.end()
- in2 points to 5 in v2
- out points to output.end()

Previous resolution [SUPERSEDED]:

This wording is relative to [N5032](#).

1. Modify 26.8.7.4 [\[set.intersection\]](#) as indicated:

```

[...]  
template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,  
        weakly_incrementable O, class Comp = ranges::less,  
        class Proj1 = identity, class Proj2 = identity>  
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>  
constexpr ranges::set_intersection_result<I1, I2, O>  
    ranges::set_intersection(I1 first1, S1 last1, I2 first2, S2 last2, O result,  
                           Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});  
template<input_range R1, input_range R2, weakly_incrementable O,  
        class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>  
requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>  
constexpr ranges::set_intersection_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, O>

```

```

        ranges::set_intersection(R1&& r1, R2&& r2, 0 result,
                                Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution-policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        random_access_iterator O, sized_sentinel_for<O> OutS, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
ranges::set_intersection_result<I1, I2, O>
    ranges::set_intersection(Ep&& exec, I1 first1, S1 last1,
                            I2 first2, S2 last2, 0 result, OutS result_last,
                            Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution-policy Ep, sized-random-access-range R1, sized-random-access-range R2,
        sized-random-access-range OutR, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp, Proj1, Proj2>
ranges::set_intersection_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>,
                                borrowed_iterator_t<OutR>>
    ranges::set_intersection(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r, Comp comp = {},
                            Proj1 proj1 = {}, Proj2 proj2 = {});

```

-1- Let: [...]

-2- *Preconditions*: The ranges [first1, last1) and [first2, last2) are sorted with respect to comp and proj1 or proj2, respectively. The resulting range does not overlap with either of the original ranges.

-3- *Effects*: Constructs a sorted intersection of the elements from the two ranges; that is, the set of elements that are present in both of the ranges. If [first1, last1) contains m elements that are equivalent to each other and [first2, last2) contains n elements that are equivalent to them, the first $\min(m, n)$ elements from the first range are included in the sorted intersection. If, of those elements, k elements from the first range are copied to the output range, then the first k elements from the second range are considered *skipped*. If $N < M$, a non-copied element is also considered skipped if it compares less than the $(N + 1)^{\text{th}}$ element of the sorted intersection. Copies the first N elements of the sorted intersection to the range [result, result + N).

-4- *Returns*:

(4.1) — result_last for the overloads in namespace std.

(4.2) — {last1, last2, result + N } for the [non-parallel algorithm](#) overloads in namespace ranges, if N is equal to M .

(4.?) — {first1 + A , first2 + B , result + N } for the [parallel algorithm](#) overloads in namespace ranges, if N is equal to M and where A and B are the numbers of copied or skipped elements in [first1, last1) and [first2, last2), respectively.

(4.3) — Otherwise, {first1 + A , first2 + B , result_last} for the overloads in namespace ranges, where A and B are the numbers of copied or skipped elements in [first1, last1) and [first2, last2), respectively.

[Croydon 2026-03-24; update wording after LWG review]

[Croydon 2026-03-24; move to Immediate]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 26.8.7.4 [\[set.intersection\]](#) as indicated:

```

[...]
template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        weakly_incrementable O, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
constexpr ranges::set_intersection_result<I1, I2, O>
    ranges::set_intersection(I1 first1, S1 last1, I2 first2, S2 last2, 0 result,
                            Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<input_range R1, input_range R2, weakly_incrementable O,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
constexpr ranges::set_intersection_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, O>
    ranges::set_intersection(R1&& r1, R2&& r2, 0 result,
                            Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution-policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        random_access_iterator O, sized_sentinel_for<O> OutS, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
ranges::set_intersection_result<I1, I2, O>
    ranges::set_intersection(Ep&& exec, I1 first1, S1 last1,
                            I2 first2, S2 last2, 0 result, OutS result_last,
                            Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution-policy Ep, sized-random-access-range R1, sized-random-access-range R2,
        sized-random-access-range OutR, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>

```

```
requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp, Proj1, Proj2>
ranges::set_intersection_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>,
    borrowed_iterator_t<OutR>>
    ranges::set_intersection(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r, Comp comp = {},
        Proj1 proj1 = {}, Proj2 proj2 = {});
```

-1- Let: [...]

-2- *Preconditions*: The ranges [first1, last1) and [first2, last2) are sorted with respect to comp and proj1 or proj2, respectively. The resulting range does not overlap with either of the original ranges.

-3- *Effects*: Constructs a sorted intersection of the elements from the two ranges; that is, the set of elements that are present in both of the ranges. If [first1, last1) contains m elements that are equivalent to each other and [first2, last2) contains n elements that are equivalent to them, the first $\min(m, n)$ elements from the first range are included in the sorted intersection. If, of those elements, k elements from the first range are copied to the output range, then the first k elements from the second range are considered *skipped*. ~~If $N < M$, a~~ A non-copied element is also considered skipped if it compares less than the $\min(M, N + 1)^{\text{th}}$ ~~$(N + 1)^{\text{th}}$~~ element of the sorted intersection. Copies the first N elements of the sorted intersection to the range [result, result + N).

-4- *Returns*:

(4.1) — result_last for the overloads in namespace std.

(4.2) — {last1, last2, result + N } for the **non-parallel algorithm** overloads in namespace ranges, ~~if N is equal to M .~~

(4.3) — Otherwise, {first1 + A , first2 + B , ~~result_last~~ **result + N** } for the **parallel algorithm** overloads in namespace ranges, where A and B are the numbers of copied or skipped elements in [first1, last1) and [first2, last2), respectively.

4549. vprint_nonunicode_buffered ignores its stream parameter

Section: 31.7.10 [print.fun] **Status:** [Immediate](#) **Submitter:** Abhinav Agarwal **Opened:** 2026-03-21 **Last modified:** 2026-03-25

Priority: Not Prioritized

View all other issues in [print.fun].

Discussion:

The *Effects* element of vprint_nonunicode_buffered(FILE* stream, string_view fmt, format_args args) in 31.7.10 [print.fun] p14 is specified as:

```
string out = vformat(fmt, args);
vprint_nonunicode("{} ", make_format_args(out));
```

That call selects the two-argument overload vprint_nonunicode(string_view, format_args) (31.7.10 [print.fun] p13), whose *Effects* element delegates to vprint_nonunicode(stdout, fmt, args). The caller-supplied stream is discarded and the output destination becomes stdout.

The parallel function vprint_unicode_buffered (31.7.10 [print.fun] p8) correctly passes stream:

```
string out = vformat(fmt, args);
vprint_unicode(stream, "{} ", make_format_args(out));
```

This is user-visible: print(FILE* stream, ...) (31.7.10 [print.fun] p2) calls vprint_nonunicode_buffered(stream, ...) on the non-UTF-8 buffered path, so the current wording routes output to the wrong stream.

[Croydon 2026-03-24; move to Immediate]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 31.7.10 [print.fun] as indicated:

```
void vprint_nonunicode_buffered(FILE* stream, string_view fmt, format_args args);
```

-14- *Effects*: Equivalent to:

```
string out = vformat(fmt, args);
vprint_nonunicode(stream, "{} ", make_format_args(out));
```

4550. Need new feature test macros for <stdckdint.h> and <stdbit.h>

Section: 17.3.2 [version.syn] **Status:** [Immediate](#) **Submitter:** Jonathan Wakely **Opened:** 2026-03-23 **Last modified:** 2026-03-24

Priority: Not Prioritized

View other active issues in [version.syn].

View all other issues in [version.syn].

Discussion:

Addresses [GB 07](#) and [GB 11](#).

The `__STDC_VERSION__` macro defined by C is not sufficient to check if a C++26 implementation provides a usable `<stdckdint.h>` header. If a `libc` header of that name is found somewhere in the C++ compiler's include paths (e.g. in `/usr/include`) then it might be included and would define the macro. However, the `libc` version of the header might use C-specific features such as `_Bool` or `_Generic` and cause errors in a C++ program. A C++-specific macro would only be present in a C++-aware header, and so a C++ program could check if that C++-specific macro is defined in `<version>` to be sure that the header is usable in C++ code.

The same argument applies to `<stdbit.h>`.

[2026-03-23 Croydon; move to Immediate]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 17.3.2 [\[version.syn\]](#) as indicated:

```
#define __cpp_lib_starts_ends_with      201711L // also in <string>, <string_view>
#define __cpp_lib_stdatomic_h          202011L // also in <stdatomic.h>
#define __cpp_lib_stdbit_h             20YYMML // also in <stdbit.h>
#define __cpp_lib_stdckdint_h          20YYMML // also in <stdckdint.h>
#define __cpp_lib_string_contains      202011L // also in <string>, <string_view>
#define __cpp_lib_string_resize_and_overwrite 202110L // also in <string>
```

4552. `compare_exchange_weak` writes a value on spurious failure, not memory contents

Section: 32.5.7.2 [\[atomics.ref.ops\]](#), 32.5.8.2 [\[atomics.types.operations\]](#) Status: [Immediate](#) Submitter: Jonathan Wakely Opened: 2026-03-24 Last modified: 2026-03-27

Priority: 3

View other [active issues](#) in [\[atomics.ref.ops\]](#).

View all other [issues](#) in [\[atomics.ref.ops\]](#).

Discussion:

[\[atomic.ref.ops\]](#) says:

-27- *Remarks:* A weak compare-and-exchange operation may fail spuriously. That is, even when the contents of memory referred to by `expected` and `ptr` are equal, it may return `false` and store back to `expected` the same memory contents that were originally there.

I think this is missing an update from [P0528R3](#) which should have changed "the same memory contents" to something like "the same value". When the comparison succeeds but the weak CAS fails anyway, the bits of exchange are replaced by the bits copied from `*ptr`. Those bits might differ from the original bits of `expected` in their padding.

[Croydon 2026-03-24; Set priority to 2 and send to SG1.]

[Croydon 2026-03-27; SG1 approved the change.]

[Croydon 2026-03-27; move to Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 32.5.7.2 [\[atomics.ref.ops\]](#) as indicated:

```
constexpr bool compare_exchange_weak(value_type& expected, value_type desired,
                                     memory_order order = memory_order::seq_cst) const noexcept;

constexpr bool compare_exchange_strong(value_type& expected, value_type desired,
                                       memory_order order = memory_order::seq_cst) const noexcept;
```

-23- *Constraints:* `is_const_v<T>` is false.

-24- *Preconditions:* failure is `memory_order::relaxed`, `memory_order::acquire`, or `memory_order::seq_cst`.

-25- *Effects:* Retrieves the value in `expected`. It then atomically compares the value representation of the value referenced by `*ptr` for equality with that previously retrieved from `expected`, and if true, replaces the value referenced by `*ptr` with that in `desired`. If and only if the comparison is true, memory is affected according to the value of success, and if the comparison is false, memory is affected according to the value of failure. When only one `memory_order` argument is supplied, the value of success is `order`, and the value of failure is `order` except that a value of `memory_order::acq_rel` shall be replaced by the value `memory_order::acquire` and a value of `memory_order::release` shall be replaced by the value `memory_order::relaxed`. If and only if the comparison is false then, after the atomic operation, the value in `expected` is replaced by the value read from the value referenced by `*ptr` during the atomic comparison. If the operation returns true, these operations are atomic read-modify-write operations (6.10.2.2 [\[intro.races\]](#)) on the value referenced by `*ptr`. Otherwise, these operations are atomic load operations on that memory.

-26- *Returns:* The result of the comparison.

-27- *Remarks:* A weak compare-and-exchange operation may fail spuriously. That is, even when the ~~contents of memory~~ **value representations** referred to by `expected` and `ptr` ~~are~~ **compare** equal, it may return false and store back to `expected` the same ~~memory contents that were~~ **value representation that was** originally there.

[*Note 2:* This spurious failure enables implementation of compare-and-exchange on a broader class of machines, e.g., load-locked store-conditional machines. A consequence of spurious failure is that nearly all uses of weak compare-and-exchange will be in a loop. When a compare-and-exchange is in a loop, the weak version will yield better performance on some platforms. When a weak compare-and-exchange would require a loop and a strong one would not, the strong one is preferable. — *end note*]

2. Modify 32.5.8.2 [\[atomics.types.operations\]](#) as indicated:

```
bool compare_exchange_weak(T& expected, T desired,
                           memory_order success, memory_order failure) volatile noexcept;
constexpr bool compare_exchange_weak(T& expected, T desired,
                                     memory_order success, memory_order failure) noexcept;
bool compare_exchange_strong(T& expected, T desired,
                             memory_order success, memory_order failure) volatile noexcept;
constexpr bool compare_exchange_strong(T& expected, T desired,
                                       memory_order success, memory_order failure) noexcept;
bool compare_exchange_weak(T& expected, T desired,
                           memory_order order = memory_order::seq_cst) volatile noexcept;
constexpr bool compare_exchange_weak(T& expected, T desired,
                                     memory_order order = memory_order::seq_cst) noexcept;
bool compare_exchange_strong(T& expected, T desired,
                             memory_order order = memory_order::seq_cst) volatile noexcept;
constexpr bool compare_exchange_strong(T& expected, T desired,
                                       memory_order order = memory_order::seq_cst) noexcept;
```

-21- *Constraints:* For the volatile overload of this function, `is_always_lock_free` is true.

-22- *Preconditions:* failure is `memory_order::relaxed`, `memory_order::acquire`, or `memory_order::seq_cst`.

-25- *Effects:* Retrieves the value in `expected`. It then atomically compares the value representation of the value pointed to by this for equality with that previously retrieved from `expected`, and if true, replaces the value pointed to by this with that in `desired`. If and only if the comparison is true, memory is affected according to the value of success, and if the comparison is false, memory is affected according to the value of failure. When only one `memory_order` argument is supplied, the value of success is `order`, and the value of failure is `order` except that a value of `memory_order::acq_rel` shall be replaced by the value `memory_order::acquire` and a value of `memory_order::release` shall be replaced by the value `memory_order::relaxed`. If and only if the comparison is false then, after the atomic operation, the value in `expected` is replaced by the value read from the value pointed to by this during the atomic comparison. If the operation returns true, these operations are atomic read-modify-write operations ([6.10.2 \[intro.multithread\]](#), [6.10.2.2 \[intro.races\]](#)) on the value pointed to by this. Otherwise, these operations are atomic load operations on that memory.

-24- *Returns:* The result of the comparison.

-25- [*Note 4:* For example, the effect of `compare_exchange_strong` on objects without padding bits (6.9.1 [\[basic.types.general\]](#)) is

```
if (memcmp(this, &expected, sizeof(*this)) == 0)
    memcpy(this, &desired, sizeof(*this));
else
    memcpy(&expected, this, sizeof(*this));
```

— *end note*]

[*Example 1:* The expected use of the compare-and-exchange operations is as follows. The compare-and-exchange operations will update `expected` when another iteration of the loop is needed.

```
expected = current.load();
do {
    desired = function(expected);
} while (!current.compare_exchange_weak(expected, desired));
```

— *end example*]

[*Example 2:* Because the expected value is updated only on failure, code releasing the memory containing the expected value on success will work. For example, list head insertion will act atomically and would not introduce a data race in the following code:

```
do {
    p->next = head;
} while (!head.compare_exchange_weak(p->next, p)); // make new list node point to the current head // try to insert
```

— *end example*]

-26- **Recommended practice:** Implementations should ensure that weak compare-and-exchange operations do not consistently return false unless either the atomic object has value different from `expected` or there are concurrent modifications to the atomic object.

-27- *Remarks:* A weak compare-and-exchange operation may fail spuriously. That is, even when the ~~contents of memory~~ **value representations** referred to by `expected` and this ~~are~~ **compare** equal, it may return false and store back to `expected` the same ~~memory contents that were~~ **value representation that was** originally there.

[*Note 2:* This spurious failure enables implementation of compare-and-exchange on a broader class of machines, e.g., load-locked store-conditional machines. A consequence of spurious failure is that nearly all uses of weak compare-and-exchange will be in a loop. When a compare-and-exchange is in a loop, the weak version will yield better performance on some platforms. When a weak compare-and-exchange would require a loop and a strong one would not, the strong one is preferable. — *end note*]

[4553](#). Wording for FR-025-246 25.7.18.2 Add a `reserve_hint` function to `concat_view`

Section: 25.7.18.2 [[range.concat.view](#)] Status: [Immediate](#) Submitter: Hui Xie Opened: 2026-03-24 Last modified: 2026-03-25

Priority: Not Prioritized

View other [active issues](#) in [[range.concat.view](#)].

View all other [issues](#) in [[range.concat.view](#)].

Discussion:

Addresses [FR-025-246](#)

LEWG approved the NB comment "FR-025-246 25.7.18.2 Add a `reserve_hint` function to `concat_view`" without wording. This issue proposes the wording.

[Croydon 2026-03-24; move to Immediate]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 25.7.18.2 [[range.concat.view](#)] as indicated:

```
namespace std::ranges {
    [...]
    template<input_range... Views>
        requires (view<Views> && ...) && (sizeof...(Views) > 0) &&
            concatable<Views...>
    class concat_view : public view_interface<concat_view<Views...>> {
    [...]
        constexpr auto size() requires (sized_range<Views> && ...);
        constexpr auto size() const requires (sized_range<const Views> && ...);
        constexpr auto reserve hint() requires (approximately sized range<Views> && ...);
        constexpr auto reserve hint() const requires (approximately sized range<const Views> && ...);
    };
}
[...]
```

```
constexpr auto size() requires (sized_range<Views> && ...);
constexpr auto size() const requires (sized_range<const Views> && ...);
```

-8- Effects: Equivalent to:

```
return apply(
    [](auto... sizes) {
        using CT = make_unsigned_like_t<common_type_t<decltype(sizes)...>>;
        return (CT(sizes) + ...);
    },
    tuple_transform(ranges::size, views_));
```

```
constexpr auto reserve hint() requires (approximately sized range<Views> && ...);
constexpr auto reserve hint() const requires (approximately sized range<const Views> && ...);
```

-9- Effects: Equivalent to:

```
return apply(
    [](auto... sizes) {
        using CT = make_unsigned_like_t<common_type_t<decltype(sizes)...>>;
        return (CT(sizes) + ...);
    },
    tuple_transform(ranges::reserve hint, views_));
```

[4554](#). Remove undefined behaviour from `hive` for invalid limits

Section: 23.3.9.1 [[hive.overview](#)] Status: [Immediate](#) Submitter: Jonathan Wakely Opened: 2026-03-24 Last modified: 2026-03-26

Priority: Not Prioritized

Discussion:

Addresses [US 139-232](#)

23.3.9.1 [[hive.overview](#)] says:

- 5- Limits can be placed on both the minimum and maximum element capacities of element blocks, both by users and implementations.

...

(5.4) — If user-specified limits are not within hard limits, or if the specified minimum limit is greater than the specified maximum limit, the behavior is undefined.

The NB comment says:

Paragraph 5.4 allows for UB when the user doesn't manage to keep the block limits within bounds and the min below the max, yet no tools are offered to check these conditions easily. In terms of providing safe facilities this UB is an easy trap to fall into. The methods that allow to set block limits on hive should rather check the conditions and throw in case of violation or at least some function should be provided to let the user check the conditions easily.

Proposed change: Adjust all constructors and methods on hive to throw in case specified block limits violate the conditions set out here. Or at least provide a function to the user to correctly check the condition separately.

[2026-03-24; Jonathan adds wording]

Could throw when the precondition is violated, or make it a *Hardened Precondition*, or ... something else? Three options are presented, which are not entirely mutually exclusive.

Previous resolution [SUPERSEDED]:

This wording is relative to [N5032](#).

Option A - make hive check for invalid limits and throw

1. Modify 23.3.9.1 [\[hive.overview\]](#) as indicated:

-5- Limits can be placed on both the minimum and maximum element capacities of element blocks, both by users and implementations.

(5.1) — The minimum limit shall be no larger than the maximum limit.

(5.2) — When limits are not specified by a user during construction, the implementation's default limits are used.

(5.3) — The default limits of an implementation are not guaranteed to be the same as the minimum and maximum possible capacities for an implementation's element blocks.

[Note 2: To allow latitude for both implementation-specific and user-directed optimization. — *end note*]

The latter are defined as hard limits. The maximum hard limit shall be no larger than `std::allocator_traits<Allocator>::max_size()`.

~~(5.4) — If user-specified limits are not within hard limits, or if the specified minimum limit is greater than the specified maximum limit, the behavior is undefined.~~

(5.5) — An element block is said to be *within the bounds* of a pair of minimum/maximum limits when its capacity is greater-or-equal-to the minimum limit and less-than-or-equal-to the maximum limit.

2. Modify 23.3.9.2 [\[hive.cons\]](#) as indicated:

```
constexpr hive(hive_limits block_limits, const Allocator&);
```

-3- *Effects*: Constructs an empty hive, using the specified allocator. Initializes *current-limits* with `block_limits`.

-4- *Complexity*: Constant.

-?- *Throws*: `invalid_argument` if `block_limits.min` or `block_limits.max` is outside the hard limits, or if `block_limits.min` is greater than `block_limits.max`.

Similar changes to other constructors and reshape

Option B - provide a helper to make limits valid

1. Modify 23.3.8 [\[hive.syn\]](#) as indicated:

```
namespace std {
    struct hive_limits {
        size_t min;
        size_t max;
        constexpr hive_limits(size_t minimum, size_t maximum) noexcept
            : min(minimum), max(maximum) {}

        constexpr hive_limits clamp_to(const hive_limits& lim) const noexcept
        { return {clamp(min, lim.min, lim.max), clamp(max, lim.min, lim.max)}; }
    };
};
```

Option C - add a helper to validate limits against hard limits and define *Preconditions* using it.

1. Modify 23.3.8 [\[hive.syn\]](#) as indicated:

```
namespace std {
    struct hive_limits {
        size_t min;
        size_t max;
        constexpr hive_limits(size_t minimum, size_t maximum) noexcept
            : min(minimum), max(maximum) {}
    };
};
```

```
constexpr bool valid_for(const hive_limits& lim) const noexcept
{ return min <= max && lim.min <= min && max <= lim.max; }
};
```

2. Modify 23.3.9.2 [hive.cons] as indicated:

```
constexpr hive(hive_limits block_limits, const Allocator&);
```

-?- *Preconditions:* block_limits.valid_for(block_capacity_hard_limits()) is true.

-3- *Effects:* Constructs an empty hive, using the specified allocator. Initializes *current-limits* with block_limits.

-4- *Complexity:* Constant.

Similar changes to other constructors and reshape

[2026-03-26; Jonathan and Frank provide new wording]

Provide a new static member function on hive which compares a hive_limits to its hard limits. Change behaviour from undefined to erroneous with implementation-defined effects.

[Croydon 2026-03-26; move to Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 23.3.9.1 [hive.overview] as indicated:

-5- Limits can be placed on both the minimum and maximum element capacities of element blocks, both by users and implementations.

(5.1) — The minimum limit shall be no larger than the maximum limit.

(5.2) — When limits are not specified by a user during construction, the implementation's default limits are used.

(5.3) — The default limits of an implementation are not guaranteed to be the same as the minimum and maximum possible capacities for an implementation's element blocks.

[Note 2: To allow latitude for both implementation-specific and user-directed optimization. — end note]

The latter are defined as hard limits. The maximum hard limit shall be no larger than `std::allocator_traits<Allocator>::max_size()`.

(5.4) — If user-specified limits *passed to a hive constructor or reshape* are not within hard limits, or if the specified minimum limit is greater than the specified maximum limit, the behavior is *undefined erroneous and the effects are implementation-defined*.

[Note ? : This condition can be checked using *is within hard limits*. — end note]

(5.5) — An element block is said to be *within the bounds* of a pair of minimum/maximum limits when its capacity is greater-or-equal-to the minimum limit and less-than-or-equal-to the maximum limit.

2. Modify 23.3.9.1 [hive.overview] as indicated:

```
constexpr hive_limits block_capacity_limits() const noexcept;
static constexpr hive_limits block_capacity_default_limits() noexcept;
static constexpr hive_limits block_capacity_hard_limits() noexcept;
static constexpr bool is_within_hard_limits(hive_limits) noexcept;
void reshape(hive_limits block_limits);
```

3. Modify 23.3.9.3 [hive.capacity] as indicated:

```
static constexpr hive_limits block_capacity_hard_limits() noexcept;
```

-19- *Returns:* A hive_limits struct with the min and max members set to the implementation's hard limits.

```
static constexpr bool is_within_hard_limits(hive_limits lim) noexcept;
```

-?- *Lets* hl be block_capacity_hard_limits().

-?- *Returns:* hl.min <= lim.min && lim.min <= lim.max && lim.max <= hl.max.

4555. Remove is_consteval_only

Section: 21.3.6.4 [meta.unary.prop], 21.4.17 [meta.reflection.traits] **Status:** [Immediate](#) **Submitter:** Tim Song **Opened:** 2026-03-24 **Last modified:** 2026-03-26

Priority: Not Prioritized

View other [active issues](#) in [meta.unary.prop].

View all other [issues](#) in [meta.unary.prop].

Discussion:

Addresses [US 81-149](#).

In light of recent EWG polls related to consteval-only types, it would be improvident to ship this trait in C++26. This also [resolves](#) the NB comment requesting respecification of its precondition.

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 21.3.3 [\[meta.type.synop\]](#), header <type_traits> synopsis, as indicated:

```
[...]
// 21.3.6.4 \[meta.unary.prop\], type properties
[...]template<class T> struct is_consteval_only;
[...]  
// 21.3.6.4 \[meta.unary.prop\], type properties
[...]template<class T>  
constexpr bool is_consteval_only_v = is_consteval_only<T>::value;
[...]
```

2. Modify [\[tab:meta.unary.prop\]](#), Table 54 — Type property predicates as indicated:

Template	Condition	Preconditions
...	...	...
template<class T> struct is_consteval_only;	T is consteval only (6.9.1 [basic.types.general])	remove_all_extents_t<T> shall be a complete type or cv-void.
...	...	...

3. Modify 21.4.1 [\[meta.syn\]](#), header <meta> synopsis, as indicated:

```
[...]
// associated with 21.3.6.4 \[meta.unary.prop\], type properties
[...]constexpr bool is_consteval_only_type(info type);
[...]
```

4. Modify 21.4.17 [\[meta.reflection.traits\]](#) as indicated:

```
[...]
// associated with 21.3.6.4 \[meta.unary.prop\], type properties
[...]constexpr bool is_consteval_only_type(info type);
[...]  
  
-3- [...]
```

4556. Unclear properties of reflection strings

Section: 21.4.1 [\[meta.syn\]](#) Status: [Immediate](#) Submitter: Tim Song Opened: 2026-03-25 Last modified: 2026-03-25

Priority: Not Prioritized

View all other [issues](#) in [\[meta.syn\]](#).

Discussion:

We are missing a statement about the lifetime and other properties of the strings returned from the reflection functions.

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 21.4.1 [\[meta.syn\]](#) as indicated:

```
-3- Any function in namespace std::meta whose return type is string_view or u8string_view returns an object V such that V.data()
[V.size()] equals '\0'. Each element of the range V.data() + [0, V.size()) is a potentially non-unique object with static storage duration
that is usable in constant expressions (6.8.2 \[intro.object\], 7.7 \[expr.const\]): a pointer to such an element is not suitable for use as a template
argument for a constant template parameter of pointer type (13.4.3 \[temp.arg.nontype\]).
```

4557. Remove constexpr from owner_less and owner_before

Section: 20.3.2.2.6 [\[util.smartptr.shared.obs\]](#), 20.3.2.3.6 [\[util.smartptr.weak.obs\]](#), 20.3.2.4 [\[util.smartptr.ownerless\]](#) Status: [Immediate](#) Submitter: Tim Song
Opened: 2026-03-25 Last modified: 2026-03-25

Priority: Not Prioritized

View all other [issues](#) in [\[util.smartptr.shared.obs\]](#).

Discussion:

Addresses [US 75-138](#)

LEWG decided to remove constexpr from owner_before and owner_less.

[Croydon 2026-03-25; move to Immediate.]

Proposed resolution:

This wording is relative to [N5032](#).

1. Modify 20.3.2.2.1 [\[util.smartptr.shared.general\]](#), class template shared_ptr synopsis, as indicated:

```
namespace std {
    template<class T> class shared_ptr {
    public:
        [...]
        template<class U>
            constexpr bool owner_before(const shared_ptr<U>& b) const noexcept;
        template<class U>
            constexpr bool owner_before(const weak_ptr<U>& b) const noexcept;
        size_t owner_hash() const noexcept;
        template<class U>
            constexpr bool owner_equal(const shared_ptr<U>& b) const noexcept;
        template<class U>
            constexpr bool owner_equal(const weak_ptr<U>& b) const noexcept;
    };
    [...]
}
```

2. Modify 20.3.2.2.6 [\[util.smartptr.shared.obs\]](#) as indicated:

```
template<class U> constexpr bool owner_before(const shared_ptr<U>& b) const noexcept;
template<class U> constexpr bool owner_before(const weak_ptr<U>& b) const noexcept;

-19- [...]
```

3. Modify 20.3.2.3.1 [\[util.smartptr.weak.general\]](#), class template weak_ptr synopsis, as indicated:

```
namespace std {
    template<class T> class weak_ptr {
    public:
        [...]
        template<class U>
            constexpr bool owner_before(const shared_ptr<U>& b) const noexcept;
        template<class U>
            constexpr bool owner_before(const weak_ptr<U>& b) const noexcept;
        size_t owner_hash() const noexcept;
        template<class U>
            constexpr bool owner_equal(const shared_ptr<U>& b) const noexcept;
        template<class U>
            constexpr bool owner_equal(const weak_ptr<U>& b) const noexcept;
    };
    [...]
}
```

4. Modify 20.3.2.3.6 [\[util.smartptr.weak.obs\]](#) as indicated:

```
template<class U> constexpr bool owner_before(const shared_ptr<U>& b) const noexcept;
template<class U> constexpr bool owner_before(const weak_ptr<U>& b) const noexcept;

-4- [...]
```

5. Modify 20.3.2.4 [\[util.smartptr.ownerless\]](#), class template owner_less synopsis, as indicated:

```
namespace std {
    template<class T = void> struct owner_less;

    template<class T> struct owner_less<shared_ptr<T>> {
        constexpr bool operator()(const shared_ptr<T>&, const shared_ptr<T>&) const noexcept;
        constexpr bool operator()(const shared_ptr<T>&, const weak_ptr<T>&) const noexcept;
        constexpr bool operator()(const weak_ptr<T>&, const shared_ptr<T>&) const noexcept;
    };

    template<class T> struct owner_less<weak_ptr<T>> {
        constexpr bool operator()(const weak_ptr<T>&, const weak_ptr<T>&) const noexcept;
        constexpr bool operator()(const shared_ptr<T>&, const weak_ptr<T>&) const noexcept;
        constexpr bool operator()(const weak_ptr<T>&, const shared_ptr<T>&) const noexcept;
    };

    template<> struct owner_less<void> {
        template<class T, class U>
            constexpr bool operator()(const shared_ptr<T>&, const shared_ptr<U>&) const noexcept;
        template<class T, class U>
            constexpr bool operator()(const shared_ptr<T>&, const weak_ptr<U>&) const noexcept;
        template<class T, class U>
            constexpr bool operator()(const weak_ptr<T>&, const shared_ptr<U>&) const noexcept;
    };
}
```

```
template<class T, class U>
    constexpr bool operator()(const weak_ptr<T>&, const weak_ptr<U>&) const noexcept;
using is_transparent = unspecified;
};
```